

Computadores com conjunto reduzido de instruções (RISC)

13.1 Características da execução de instruções

- Operações
- Operandos
- Chamadas de procedimento
- Implicações

13.2 Uso de um banco grande de registradores

- Janelas de registradores
- Variáveis globais
- Grande banco de registro *versus* cache

13.3 Otimização de registradores baseada em compiladores

13.4 Arquitetura com conjunto reduzido de instruções

- Por que CISC
- Características da arquitetura com conjuntos reduzidos de instruções
- Características CISC *versus* RISC

13.5 Pipeline no RISC

- Pipeline com instruções regulares
- Otimização de pipeline

13.6 MIPS R4000

- Conjunto de instruções
- Pipeline de instruções

13.7 SPARC

- Conjunto de registradores do SPARC
- Conjunto de instruções
- Formato da instrução

13.8 Controvérsia de RISC *versus* CISC

13.9 Leitura recomendada

PRINCIPAIS PONTOS

- Estudos do comportamento de execução dos programas de linguagem de alto nível forneceram um guia para projetar um novo tipo de arquitetura de processador: computador com conjunto reduzido de instruções (RISC — *reduced instructions set computer*). As instruções de atribuição predominam, sugerindo que os movimentos simples de dados deveriam ser otimizados. Há também muitas instruções IF e LOOP, o que sugere que o mecanismo de controle de sequências subjacentes necessita ser otimizado para permitir pipeline eficiente. Estudos sobre padrões de referência de operando sugerem que deveria ser possível melhorar o desempenho guardando um número moderado de operandos nos registradores.
- Esses estudos motivaram as principais características das máquinas RISC: (1) um conjunto de instruções limitado com um formato fixo, (2) um número grande de registradores ou o uso do compilador que otimize o uso de registradores e (3) uma ênfase na otimização do pipeline de instruções.
- O conjunto de instruções simples de uma máquina RISC leva por si só a um pipeline eficiente porque há menos operações por instrução e elas são mais previsíveis. Uma arquitetura com conjunto de instruções RISC também leva por si só à técnica de desvio atrasado, na qual instruções de desvio são rearranjadas com outras instruções para melhorar a eficiência do pipeline.

Desde o desenvolvimento de computadores que armazenavam programas nos anos 1950, houve consideravelmente poucas inovações verdadeiras nas áreas de organização e arquitetura de computadores. A seguir, temos alguns dos maiores avanços desde o nascimento do computador:

- **Conceito de família:** introduzido pela IBM com seu System/360 em 1964, seguido logo depois por DEC com seu PDP-8. O conceito de família se para a arquitetura de uma máquina da sua implementação. Um conjunto de computadores é oferecido, com características de preço/desempenho diferentes e que apresentam a mesma arquitetura ao usuário. As diferenças em preço e desempenho se devem às diferentes implementações da mesma arquitetura.

- **Unidade de controle microprogramada:** sugerida por Wilkes em 1951 e introduzida pela IBM na linha S/360 em 1964. A microprogramação facilita a tarefa de projetar e implementar a unidade de controle e fornece suporte para conceito de família.
- **Memória cache:** introduzida pela primeira vez comercialmente no S/360 Model 85 da IBM em 1968. A inserção deste elemento na hierarquia de memória melhora o desempenho consideravelmente.
- **Pipeline:** um meio para introduzir paralelismo na natureza essencialmente sequencial de um programa de instruções de máquina. Exemplos são pipeline de instruções e processamento vetorial.
- **Múltiplos processadores:** esta categoria cobre um número de organizações e objetivos diferentes.
- **Arquitetura de computadores com conjunto de instruções reduzido (RISC):** este é o foco deste capítulo.

A arquitetura RISC é um grande avanço da tendência histórica na arquitetura de processadores. Uma análise da arquitetura RISC traz à tona muitas questões importantes da organização de arquitetura de computadores.

Embora os sistemas RISC tenham sido definidos e projetados de muitas maneiras diferentes e por grupos diferentes, aqui estão os principais elementos compartilhados pela maioria dos projetos:

- Um grande número de registradores de propósito geral e/ou o uso de tecnologia de compiladores para otimizar uso de registradores.
- Um conjunto de instruções simples e limitado.
- Uma ênfase na otimização do pipeline de instruções.

A Tabela 13.1 compara vários sistemas RISC e não RISC.

Começamos este capítulo com um breve resumo de alguns resultados nos conjuntos de instruções e depois analisamos cada um dos três tópicos que acabamos de mencionar. Segue, depois, uma descrição de dois projetos RISC mais bem documentados.



13.1 Características da execução de instruções

Uma das formas mais visíveis da evolução associada a computadores são as linguagens de programação. À medida que o preço de hardware caiu, o custo relativo de software aumentou. Juntamente com isso, a carência crônica de programadores elevou os custos de software em termos absolutos. Assim, o principal custo no ciclo de vida de um sistema é o software, não o hardware. Somado ao custo e à inconveniência, está o elemento da falta

Tabela 13.1 Características de alguns processadores CISC, RISC e superescalares

	Computadores com conjuntos de instruções complexos (CISC)			Computadores com conjuntos de instruções reduzidos (RISC)		Superescalares		
Característica	IBM 370/168	VAX 11/780	Intel 80486	SPARC	MIPS R4000	Power PC	Ultra SPARC	MIPS R10000
Ano de desenvolvimento	1973	1978	1989	1987	1991	1993	1996	1996
Número de instruções	208	303	235	69	94	225		
Tamanho de instrução em bytes	2–6	2–57	1–11	4	4	4	4	4
Modos de endereçamento	4	22	11	1	1	2	1	1
Número de registradores de uso geral	16	16	8	40–520	32	32	40–520	32
Tamanho da memória de controle (Kb)	420	480	246	—	—	—	—	—
Tamanho da cache (KB)	64	64	8	32	128	16–32	32	64

de confiabilidade: é comum que programas, tanto sistemas como aplicações, continuem a apresentar novas falhas depois de anos de operação.

A resposta dos pesquisadores e da indústria foi desenvolver linguagens de programação de alto nível ainda mais poderosas e complexas. Essas linguagens de alto nível (HHLs — *high level languages*) permitem que o programador expresse algoritmos de forma mais concisa, cuide de mais detalhes e frequentemente suporte de maneira natural o uso de programação estruturada ou modelagem orientada a objetos.

Além disso, esta solução trouxe outro problema, conhecido como *diferença semântica*, a diferença entre operações fornecidas em linguagens de alto nível e aquelas fornecidas na arquitetura do computador. Sintomas dessa diferença são suspeitas de incluir ineficiências da execução, tamanho excessivo do programa de máquina e complexidade de compiladores. Os projetistas responderam com arquiteturas voltadas para acabar com essa diferença. Principais recursos incluem grandes conjuntos de instruções, dúzias de modos de endereçamento e várias instruções das linguagens de alto nível implementadas no hardware. Um exemplo deste último é a instrução de máquina CASE em VAX. Tais conjuntos de instruções complexos têm a intenção de:

- Facilitar a tarefa do programador de compiladores.
- Melhorar a eficiência da execução, porque sequências complexas de operações podem ser implementadas no microcódigo.
- Fornecer suporte para linguagens de programação de alto nível ainda mais complexas e sofisticadas.

Enquanto isso, um número de estudos foi feito durante anos para determinar as características e padrões de execução das instruções de máquina geradas a partir de programas das linguagens de alto nível. Os resultados desses estudos inspiraram alguns pesquisadores a procurar por uma abordagem diferente: mais precisamente, tornar a arquitetura que suporta linguagens de alto nível mais simples, em vez de mais complexa.

Para entender a linha de raciocínio dos defensores do RISC, começamos com uma breve revisão das características da execução de instruções. Os aspectos de interesse computacional são:

- **Operações efetuadas:** estas determinam as funções a serem efetuadas pelo processador e a sua interação com memória.
- **Operandos usados:** os tipos de operandos e a frequência do seu uso determinam a organização da memória para armazená-los e os modos de endereçamento para acessá-los.
- **Sequência da execução:** isto determina a organização e o controle do pipeline.

No restante desta seção, resumimos os resultados de uma série de estudos sobre programas das linguagens de alto nível. Todos os resultados são baseados em medições dinâmicas. Isto é, as medições são coletadas executando o programa e contando o número de vezes que algum recurso apareceu ou que alguma propriedade permaneceu verdadeira. Ao contrário disso, medições estáticas apenas fazem essas contagens no código fonte de um programa. Elas não dão nenhuma informação útil sobre desempenho, porque elas não são avaliadas em relação ao número de vezes que cada instrução é executada.



Operações

Uma série de estudos foi feita para analisar o comportamento dos programas de linguagens de programação de alto nível. A Tabela 4.8, discutida no Capítulo 4, inclui os principais resultados de vários estudos. Há uma coerência bastante boa nos resultados dessa mistura de linguagens e aplicações. Instruções de atribuição predominam, sugerindo que a simples movimentação de dados é de alta importância. Há também uma preponderância de instruções condicionais (IF, LOOP). Essas instruções são implementadas na linguagem de máquina com algum tipo de instruções de comparação e desvio. Isso sugere que o mecanismo de controle sequencial do conjunto de instruções é importante.

Esses resultados são instrutivos para os projetistas das instruções de máquina, indicando que tipo de instruções ocorre com mais frequência e, por isso, deveriam ser suportadas de uma maneira “otimizada”. No entanto, esses resultados não revelam quais instruções usam mais tempo de execução de um programa típico. Isto é, dado um programa compilado na linguagem de máquina, quais instruções na linguagem de origem causam a execução da maioria das instruções na linguagem de máquina?

Para chegar a informações neste nível, os programas de Patterson (PATTERSON e SEQUIN, 1982^a), descritos no Apêndice 4A, foram compilados em VAX, PDP-11 e Motorola 68000 para determinar o número médio de instruções

de máquina e as referências à memória por tipo de instrução. A segunda e a terceira colunas da Tabela 13.2 mostram a frequência relativa de ocorrência de várias instruções de linguagens de programação de alto nível em uma variedade de programas; os dados foram obtidos observando as ocorrências nos programas em execução, em vez de observar apenas o número de vezes que as instruções ocorrem no código fonte.

Estas estatísticas são de frequência dinâmica. Para obter os dados nas colunas quatro e cinco (avaliação das instruções de máquina), cada valor na segunda e terceira coluna é multiplicado pelo número de instruções de máquina produzidas pelo compilador. Esses resultados são então normalizados para que as colunas quatro e cinco mostrem a frequência relativa da ocorrência, com peso sendo atribuído pelo número de instruções de máquina por cada instrução da linguagem de programação de alto nível. De forma semelhante, a sexta e a sétima coluna são obtidas multiplicando-se a frequência de ocorrência de cada tipo de instrução pelo número relativo de referências de memória causadas por cada instrução. Os dados da coluna 4 até a coluna 7 mostram medidas que correspondem ao tempo que realmente foi gasto com a execução de vários tipos de instrução. Os resultados sugerem que a chamada/retorno do procedimento é a operação que consome mais tempo em programas típicos das linguagens de programação de alto nível.

Deve estar claro para o leitor o significado da Tabela 13.2. Ela indica a relação de vários tipos de instrução em uma linguagem de programação de alto nível, quando essa linguagem é compilada para uma arquitetura com um conjunto de instruções atual. Algumas outras arquiteturas poderiam produzir resultados diferentes. No entanto, este estudo produz resultados que são representativos para arquiteturas CISC atuais. Assim, esses resultados podem fornecer um guia para aqueles que estão procurando pelas maneiras mais eficientes para suportar linguagens de programação de alto nível.



Operandos

Muito menos trabalho foi feito na ocorrência de tipos de operandos, apesar da importância deste tópico. Vários aspectos são significantes.

O estudo de Patterson e Sequin, que já foi referenciado (PATTERSON e SEQUIN, 1982^a), analisou também a frequência dinâmica de ocorrência de classes de variáveis (Tabela 13.3). Os resultados, consistentes entre programas C e Pascal, mostram que a maioria das referências é de variáveis escalares simples. Além disso, mais de 80% de escalares eram variáveis locais (para o procedimento). As referências para vetores/estruturas requerem uma referência prévia para o seu índice ou ponteiro, o qual é novamente um escalar local. Assim, existe uma predominância de referências para escalares, e estas são altamente localizadas.

O estudo de Patterson analisou o comportamento dinâmico dos programas de linguagens de programação de alto nível, independentemente da arquitetura subjacente. Conforme discutido antes, é necessário lidar com arquiteturas atuais para examinar o comportamento dos programas mais a fundo. Um estudo (LUNDE, 1977^b) analisou as instruções do DEC-10 dinamicamente e concluiu que cada instrução referencia em média 0,5 operandos e 1,4 registradores. Resultados semelhantes são reportados em Huck (1983^c) para programas C, Pascal e FORTRAN em S/370, PDP-11 e VAX. É claro que essas figuras dependem muito tanto da arquitetura como do compilador, mas elas ilustram, sim, a frequência de acesso a operandos.

Tabela 13.2 Frequência dinâmica relativa ponderada das linguagens de programação de alto nível (Patterson e Sequin, 1982^a)

	Ocorrência dinâmica		Avaliação das instruções de máquina		Avaliação de referências de memória	
	Pascal	C	Pascal	C	Pascal	C
ATRIBUIÇÃO	45%	38%	13%	13%	14%	15%
LOOP	5%	3%	42%	32%	33%	26%
CHAMADA	15%	12%	31%	33%	44%	45%
IF	29%	43%	11%	21%	7%	13%
GOTO	—	3%	—	—	—	—
OUTROS	6%	1%	3%	1%	2%	1%

Tabela 13.3 Frequência dinâmica de operandos

	Pascal	C	Média
Constante inteira	16%	23%	20%
Variável escalar	58%	53%	55%
Array/Estrutura	26%	24%	25%

Estes últimos estudos sugerem a importância de uma arquitetura que leve por si só a um acesso rápido aos operandos, porque essa operação é efetuada com muita frequência. O estudo de Patterson sugere que o principal candidato para otimização é o mecanismo para armazenar e acessar variáveis locais escalares.



Chamadas de procedimento

Vimos que as chamadas e os retornos de procedimento são um aspecto dos programas de alto nível. A evidência (Tabela 13.2) sugere que essas são as operações que consomem mais tempo em programas compilados de alto nível. Assim, será lucrativo considerar as maneiras para implementar essas operações de forma eficiente. Dois aspectos são significativos: o número de parâmetros e variáveis com os quais um procedimento lida e a profundidade de aninhamento.

O estudo de Tanenbaum (1978^d) concluiu que para 98% de procedimentos chamados dinamicamente foram passados menos do que seis argumentos e que 92% deles usaram menos do que seis variáveis locais escalares. Resultados semelhantes foram reportados pela equipe RISC de Berkeley (KATEVENIS, 1983^e), conforme mostrado na Tabela 13.4. Esses resultados mostram que o número de palavras requeridas por ativação de procedimento não é grande. Os estudos reportados anteriormente indicaram que uma grande parte de referências de operandos é para variáveis locais escalares. Esses estudos mostram que essas referências são, na verdade, resumidas a poucas variáveis.

O mesmo grupo de Berkeley analisou também o padrão de chamadas e retornos de procedimentos em programas de alto nível. Eles concluíram que é raro haver uma grande sequência ininterrupta de chamadas de procedimento seguida pela correspondente sequência de retornos. Em vez disso, eles concluíram que um programa permanece confinado a uma janela relativamente estreita de profundidade de chamadas de procedimentos. Isso é ilustrado na Figura 4.21, que foi discutida no Capítulo 4. Esses resultados reforçam a conclusão de que referências de operandos são altamente localizadas.



Implicações

Vários grupos analisaram os resultados como os que acabamos de reportar e concluíram que a tentativa de criar uma arquitetura com um conjunto de instruções parecida com as linguagens de programação de alto nível não é a estratégia de projeto mais eficaz. Em vez disso, as linguagens de programação de alto nível podem ser mais bem suportadas com otimização de desempenho de recursos de programas típicos escritos em linguagem de alto nível que consomem mais tempo.

Tabela 13.4 Argumentos de procedimentos e variáveis escalares locais

Porcentagem de chamadas de procedimento executadas com	Compiladores, interpretadores e editores de texto	Pequenos programas não numéricos
> 3 argumentos	0–7%	0–5%
> 5 argumentos	0–3%	0 %
> 8 palavras de argumentos e escalares locais	1–20%	0–6%
> 12 palavras de argumentos e escalares locais	1–6%	0–3%

Generalizando a partir do trabalho de uma série de pesquisadores, surgem três elementos que caracterizam as arquiteturas RISC. Primeiro, o uso de um grande número de registradores ou o uso de um compilador para otimizar uso de registradores. A intenção disso é otimizar referências à operandos. Os estudos que acabamos de discutir mostram que existem várias referências para cada instrução de linguagem de alto nível e que existe uma grande proporção de instruções de movimentação (atribuições). Isso, junto com a localidade e a predominância de referências escalares, sugere que o desempenho pode ser melhorado com redução de referências à memória à custa de mais referências a registradores. Por causa da localidade dessas referências, um conjunto de registradores expandido parece prático.

Segundo, uma atenção cuidadosa precisa ser dedicada ao projeto de pipelines de instruções. Por causa de alta proporção de instruções de desvio e chamadas de procedimentos, um pipeline de instruções direto não será eficiente. Isso se manifesta pela grande proporção de instruções que são obtidas, mas nunca são executadas.

Finalmente, um conjunto de instruções simplificado (reduzido) é indicado. Este ponto não é óbvio como outros, mas se tornará mais claro na discussão seguinte.



13.2 Uso de um banco grande de registradores

Os resultados apresentados na Seção 13.1 salientam a conveniência de acesso rápido a operandos. Vimos que existe uma grande proporção de instruções de atribuição em programas de alto nível e muitas delas são do tipo simples $A \leftarrow B$. Também há um número significativo de acessos a operandos por instrução de um programa de alto nível. Se juntarmos esses resultados com o fato de a maioria de acessos ser para escalares locais, então podemos sugerir forte dependência do armazenamento em registradores.

A razão pela qual o armazenamento em registradores é indicado é que esse é o dispositivo de armazenamento mais rápido disponível, mais rápido que a memória principal e que a cache. O banco de registradores é fisicamente pequeno, no mesmo chip como a ALU e a unidade de controle, e emprega endereços bem menores do que os endereços para memória e cache. Assim, é necessária uma estratégia que irá permitir que operandos acessados mais frequentemente sejam guardados em registradores, minimizando operações registrador-memória.

Duas abordagens básicas são possíveis, uma baseada em software e outra baseada em hardware. A abordagem em software depende do compilador para maximizar o uso de registradores. O compilador tentará alocar registradores para aquelas variáveis que serão mais usadas em um determinado período. Esta abordagem requer o uso de algoritmos sofisticados para análise de programas. A abordagem de hardware é simplesmente usar mais registradores para que mais variáveis possam ser guardadas em registradores por mais tempo.

Nesta seção discutiremos a abordagem de hardware. Esta abordagem foi primeiramente utilizada pelo grupo RISC de Berkeley (Patterson e Sequin, 1982^o); foi usada no primeiro produto RISC comercial, Pyramid (RAGAN-KELLEY e CLARK, 1983^o); e é, atualmente, usada na popular arquitetura SPARC.



Janelas de registradores

Em face disto, o uso de um conjunto grande de registradores deveria diminuir a necessidade de acessar a memória. A tarefa de projeto é organizar os registradores de tal maneira que esse objetivo seja alcançado.

Como a maioria das referências de operandos é para escalares locais, a abordagem óbvia é armazená-los em registradores, com talvez alguns registradores reservados para variáveis globais. O problema é que a definição do que é *local* muda a cada chamada e retorno de procedimento e com operações que ocorrem frequentemente. A cada chamada, variáveis locais precisam ser salvas dos registradores para memória, para que os registradores possam ser reutilizados pelo programa chamado. Além disso, parâmetros precisam ser passados. No retorno, as variáveis do programa-pai devem ser restauradas (carregadas de volta em registradores) e os resultados devem ser passados de volta para o programa-pai.

A solução é baseada em outros dois resultados vistos na Seção 13.1. Primeiro, um procedimento típico emprega apenas alguns parâmetros passados e variáveis locais (Tabela 13.4). Segundo, a profundidade da ativação do procedimento flutua dentro de um intervalo relativamente estreito (Figura 4.21). Para explorar essas propriedades, vários conjuntos pequenos de registradores são usados, sendo cada um atribuído para um procedimento diferente. Uma chamada de procedimento direciona automaticamente o processador para usar uma janela de registradores

diferentes e de tamanho fixo, em vez de salvar os registradores na memória. Janelas para procedimentos adjacentes são sobrepostas para permitir passagem de parâmetros.

O conceito é ilustrado na Figura 13.1. A qualquer momento, apenas uma janela de registradores é visível e endereçável, como se fosse o único conjunto de registradores (por exemplo, endereços de 0 até $N - 1$). A janela é dividida em três áreas de tamanho fixo. Registradores de parâmetros guardam parâmetros passados a partir do procedimento que chamou o procedimento atual e guardam os resultados a serem passados de volta. Registradores locais são usados para variáveis locais, conforme atribuído pelo compilador. Registradores temporários são usados para trocar parâmetros e resultados com o próximo nível abaixo (procedimentos chamados pelo procedimento atual). Os registradores temporários em um nível são fisicamente os mesmos que os registradores de parâmetro no próximo nível abaixo. Essa sobreposição permite que os parâmetros sejam passados sem o movimento real de dados. Tenha em mente que, exceto para sobreposição, os registradores em dois níveis diferentes são fisicamente distintos. Isto é, os registradores de parâmetros e os registradores locais no nível J são separados dos registradores locais e de parâmetros do nível $J + 1$.

Para poder lidar com qualquer padrão possível de chamadas e retornos, o número de janelas de registradores teria que ser ilimitado. Em vez disso, as janelas de registradores podem ser usadas para guardar algumas ativações de procedimentos mais recentes. Ativações mais antigas têm que ser salvas em memória e restauradas depois — quando a profundidade de aninhamento diminui. Assim, a verdadeira organização do banco de registradores é como um buffer circular de janelas sobrepostas. Dois exemplos importantes desta abordagem são a arquitetura SPARC da SUN, descritas na Seção 13.7, e a arquitetura IA-64 usada no processador Itanium da Intel, descrita no Capítulo 21.

A organização circular é mostrada na Figura 13.2 que ilustra um buffer circular de seis janelas. O buffer é preenchido até uma profundidade de 4 (A chama B; B chama C; C chama D) com o procedimento estando ativo. O ponteiro da janela atual (PJA) aponta para a janela do procedimento atualmente ativo. Registradores referenciados por uma instrução de máquina são deslocados por esse ponteiro para determinar o registrador físico atual. O ponteiro da janela salva (PJS) identifica a janela mais recentemente salva na memória. Se o procedimento D agora chamar procedimento E, os argumentos para E são armazenados nos registradores temporários de D (sobreposição entre $w3$ e $w4$) e o PJA é avançado por uma janela.

Se o procedimento E então fizer uma chamada para o procedimento F, a chamada não poderá ser feita com o *status* atual do buffer. Isso acontece porque janela F sobrepõe janela A. Se F começar a carregar seus registradores temporários, preparatórios para uma chamada, ele irá sobrescrever registradores de parâmetros de A (A.in). Assim, quando PJA é incrementado (módulo 6) e se torna igual a PJS, ocorre uma interrupção e a janela A é salva. Apenas duas primeiras partes (A.in e A.loc) precisam ser salvas. Depois, o PJS é incrementado e o chamado para F procede. Uma interrupção semelhante acontece nos retornos. Por exemplo, subsequente à chamada F, quando B retorna para A, PJA é decrementado e se torna igual ao PJS. Isso causa uma interrupção que resulta em restauração da janela A.

Pode-se concluir, com base na explanação anterior, que um arquivo de registradores com N janelas pode guardar apenas $N - 1$ ativações de procedimentos. O valor de N não precisa ser grande. Conforme mencionado no Apêndice 4A, um estudo (Tamir e Sequin, 1983⁹) concluiu que, com 8 janelas, salvar ou restaurar torna-se necessário em apenas 1% de chamadas ou retornos. Computadores RISC de Berkeley usam 8 janelas com 16 registradores cada uma. Computadores Pyramid possuem 16 janelas de 32 registradores cada uma.

Figura 13.1 Sobreposição das janelas de registradores

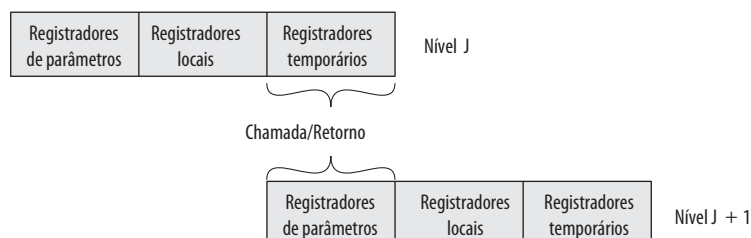
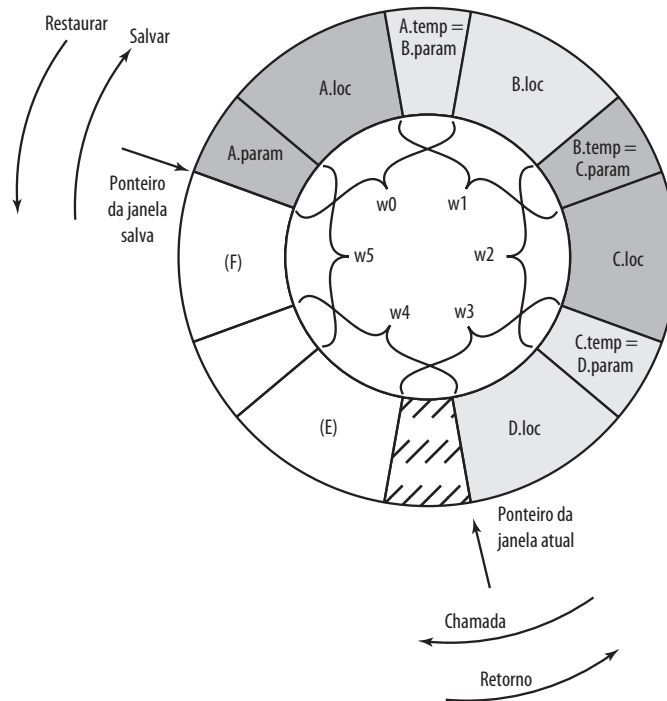


Figura 13.2 Organização do buffer circular de janelas sobrepostas

Variáveis globais

O esquema de janelas que acabamos de descrever oferece uma organização eficiente para armazenar variáveis em registradores. No entanto, este esquema não atende à necessidade de armazenar variáveis globais, aquelas acessadas por mais de um procedimento. Duas opções são sugeridas por si só. Primeiro, às variáveis declaradas como globais em uma linguagem de alto nível podem ser atribuídas posições de memória pelo compilador e todas as instruções de máquina que referenciam essas variáveis usarão operandos referenciados em memória. Isso é bem direto, do ponto de vista de hardware e de software (compilador). No entanto, para variáveis globais acessadas frequentemente, este esquema é ineficiente.

Uma alternativa é incorporar um conjunto de registradores globais no processador. Esses registradores seriam em número fixo e estariam disponíveis para todos os procedimentos. Um esquema uniforme de numeração pode ser usado para simplificar o formato da instrução. Por exemplo, referências para registradores de 0 a 7 poderiam se referir a registradores globais únicos e referências para registradores de 8 a 31 poderiam ser o deslocamento para se referir a registradores físicos na janela atual. Existe uma sobrecarga no hardware para acomodar a divisão no endereçamento de registradores. Além disso, o compilador precisa decidir que variáveis globais devem ser atribuídas aos registradores.



Grande banco de registradores *versus* cache

O arquivo de registradores, organizado em janelas, age como um pequeno buffer para guardar um subconjunto de variáveis que têm mais probabilidade de ser bastante usadas. Desse ponto de vista, o arquivo de registradores age como uma memória cache, embora uma memória muito mais rápida. A questão que aparece neste ponto é se seria mais simples e melhor usar uma cache ou um banco de registradores pequeno e tradicional.

A Tabela 13.5 compara as características das duas abordagens. O banco de registradores baseado em janelas guarda todas as variáveis locais escalares (exceto em raros casos de sobrecarga da janela) de $N - 1$ mais recentes

Tabela 13.5 Características do arquivo grande de registradores e organizações de cache

Grandes arquivos de registradores	Cache
Todas variáveis locais escalares	Variáveis locais recentemente usadas
Variáveis individuais	Blocos de memória
Variáveis globais assinaladas pelo compilador	Variáveis globais recentemente usadas
Salvar/restaurar baseados na profundidade de aninhamento do procedimento	Salvar/restaurar baseado em algoritmos de atualização da cache
Endereçamento de registrador	Endereço de memória

ativações de procedimentos. A cache guarda uma seleção de variáveis escalares recentemente usadas. O arquivo de registradores deveria economizar tempo, porque todas as variáveis locais escalares são mantidas. Por outro lado, a cache pode fazer uso mais eficiente do espaço, porque ela reage dinamicamente a situações. Além disso, a cache geralmente trata todas as referências de memória da mesma forma, incluindo instruções e outros tipos de dados. Assim, salvamentos nessas outras áreas são possíveis com uso de cache e não são possíveis com um arquivo de registradores.

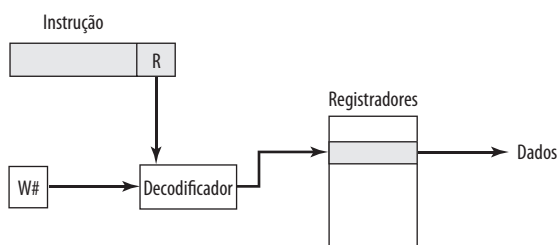
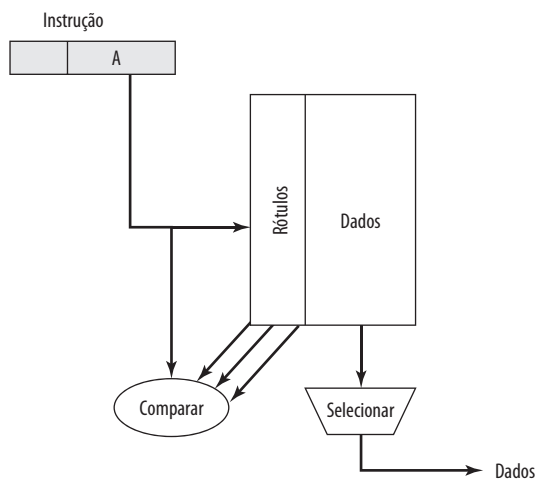
Um banco de registradores pode fazer uso ineficiente de espaço, porque nem todos os procedimentos vão precisar do espaço inteiro da janela dedicado somente para eles. Por outro lado, a cache sofre de outro tipo de ineficiência: os dados são lidos da cache em blocos. Enquanto o banco de registradores contém apenas as variáveis em uso, a cache lê um bloco de dados do qual uma parte menor ou até maior não será usada.

A cache é capaz de lidar bem com variáveis globais e locais. Existem normalmente muitos escalares globais, mas apenas alguns deles são muito usados (Katevenis, 1983^o). A cache descobrirá dinamicamente essas variáveis e irá guardá-las. Se um banco de registradores baseado em janelas for acrescido de registradores globais, ele também pode guardar alguns escalares globais. No entanto, é difícil para um compilador determinar quais variáveis globais serão muito usadas.

Com banco de registradores, o movimento de dados entre registradores e memória é determinado pela profundidade de aninhamento do procedimento. Como essa profundidade normalmente se encontra dentro de um intervalo pequeno, o uso da memória é relativamente infrequente. A maioria das memórias cache possui tamanho pequeno. Assim, existe o perigo de que outros dados ou instruções sobrescrevam variáveis frequentemente usadas.

Com base na discussão até agora, a escolha entre um banco grande de registradores baseado em janelas e uma cache não está totalmente clara. No entanto, existe uma característica pela qual a abordagem de registrador é claramente superior e que sugere que um sistema baseado em cache será sensivelmente mais lento. Essa distinção aparece na quantidade de *overhead* de endereçamento experienciada pelas duas abordagens.

A Figura 13.3 ilustra a diferença. Para referenciar um escalar local em um arquivo de registradores baseado em janelas, um número de registrador virtual e um número de janela são usados. Esses podem passar por um decodificador relativamente simples para selecionar um dos registradores físicos. Para referenciar uma posição de memória na cache, um endereço de memória de tamanho completo deve ser gerado. A complexidade desta operação depende do modo de endereçamento. Em uma cache associativa por conjunto, uma parte do endereço é usada para ler um número de palavras e rótulos iguais ao tamanho do conjunto. Outra parte do endereço é comparada com as marcações e uma das palavras que foram lidas é selecionada. Deve estar claro que, mesmo que a cache fosse tão rápida quanto o arquivo de registradores, o tempo de acesso seria consideravelmente maior. Assim, do ponto de vista de desempenho, o banco de registradores baseado em janelas é superior para escalares locais. Outras melhorias de desempenho podem ser alcançados com a adição de uma cache apenas para instruções.

Figura 13.3 Referenciando um escalar**(a) Banco de registradores baseado em janelas****(b) Cache**

13.3 Otimização de registradores baseada em compiladores

Vamos supor agora que apenas um número pequeno (16 a 32, por exemplo) de registradores esteja disponível na máquina RISC alvo. Neste caso, o uso otimizado de registradores é de responsabilidade do compilador. Um programa escrito em uma linguagem de alto nível não possui, obviamente, nenhuma referência explícita a registradores. Em vez disso, as grandezas do programa são referenciadas simbolicamente. O objetivo do compilador é guardar os operandos nos registradores durante o máximo de operações possível em vez de usar memória e minimizar operações de carregar-e-armazenar.

Em geral, a abordagem usada é a seguinte. Cada grandeza do programa que é candidata a residir em um registrador é atribuída para um registrador simbólico ou virtual. O compilador, então, mapeia o número ilimitado de registradores simbólicos para um número fixo de registradores reais. Se, em uma determinada parte do programa, houver mais grandezas para tratar do que registradores reais, então algumas das grandezas são atribuídas para posições de memória. As instruções de carregar-e-armazenar são usadas para posicionar as grandezas nos registradores temporariamente para operações computacionais.

A essência da tarefa de otimização é decidir quais grandezas devem ser atribuídas para registradores em qualquer ponto do programa. A técnica mais comumente usada em compiladores RISC é conhecida como coloração de grafos, uma técnica emprestada da disciplina de topologia (CHAITIN, 1982^h, CHOW et al., 1986ⁱ, COUTANT, HAMMOND e KELLEY, 1986^j, CHOW e HENNESSY, 1990^k).

O problema de coloração de grafos é este: dado um grafo que consiste de nós e bordas, atribuir cores para nós de tal forma que nós adjacentes tenham cores diferentes e que sejam usadas menos cores possíveis. Este problema é adaptado ao problema do compilador da seguinte maneira. Primeiramente, o programa é analisado para construir

um grafo de interferência entre registradores. Os nós do grafo são registradores simbólicos. Se dois registradores simbólicos estão “vivos” durante o mesmo fragmento do programa, então eles são unidos por uma linha para demonstrar uma interferência. Uma tentativa é feita então para colorir o grafo com n cores, onde n é o número de registradores. Nós que compartilham a mesma cor podem ser atribuídos para o mesmo registrador. Se este processo não completar totalmente, então esses nós que não podem ser coloridos devem ser colocados em memória, e cargas e armazenamentos devem ser usados para criar espaço para grandezas afetadas quando elas forem necessárias.

A Figura 13.4 é um exemplo simples do processo. Admita um programa com seis registradores simbólicos para ser compilado em três registradores reais. A Figura 13.4a mostra a sequência de tempo de uso ativo de cada registrador simbólico. Linhas horizontais pontilhadas indicam execuções sucessivas de instruções. A Figura 13.4b mostra o grafo de interferência de registradores (sombreamentos e listras são usados no lugar de cores). Uma coloração possível com três cores é mostrada. Como os registradores A e D não interferem, o compilador pode atribuir ambos ao registrador físico R1. De forma semelhante, registradores simbólicos C e E podem ser atribuídos ao registrador R3. Um registrador simbólico, F, é deixado sem cor e deve ser tratado com leitura e escrita em memória.

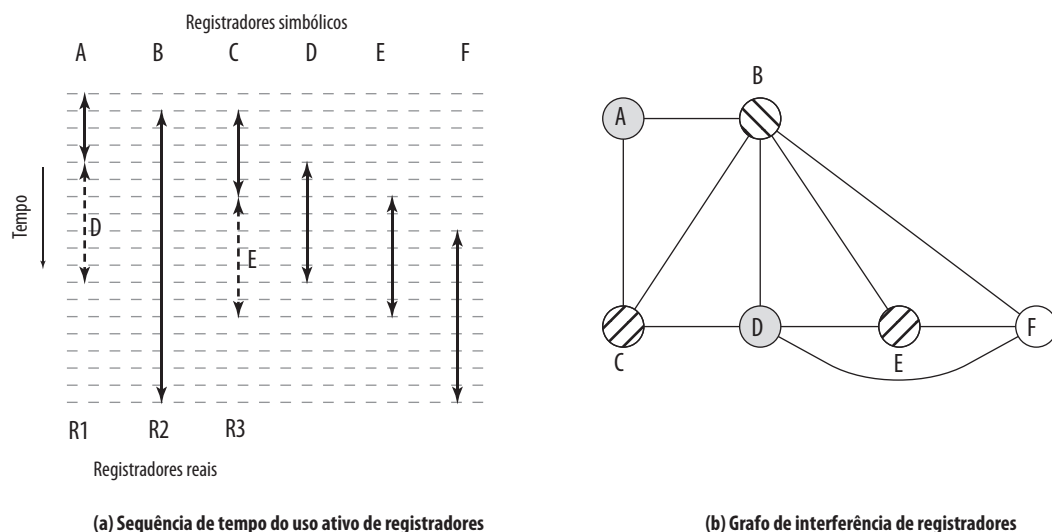
Em geral, há uma troca entre o uso de um conjunto grande de registradores e a otimização de registradores baseada em compiladores. Por exemplo, Bradlee, Eggers e Henry (1991^l) mostram em um estudo que modelou uma arquitetura RISC com recursos similares do Motorola 88000 e do MIPS R2000. Os pesquisadores variaram o número de registradores de 16 a 128 e consideraram o uso tanto de registradores de uso geral como de registradores divididos entre uso inteiro e de ponto flutuante. O seu estudo mostrou que mesmo com uma pequena otimização de registradores, há um benefício pequeno no uso de mais de 64 registradores. Com técnicas de otimização de registradores razoavelmente sofisticadas, há apenas uma melhoria mínima com mais de 32 registradores. Finalmente, eles observaram que, com um número pequeno de registradores (por exemplo, 16), uma máquina com uma organização de registradores compartilhada executa mais rapidamente do que uma com organização dividida. Conclusões semelhantes podem ser tiradas de Huguet e Lang (1991^m) que reportam um estudo preocupado, em primeiro lugar, com a otimização de um número pequeno de registradores em vez de comparar conjuntos de registradores grandes com técnicas de otimização.



13.4 Arquitetura com conjunto reduzido de instruções

Nesta seção analisamos algumas características gerais e as motivações de uma arquitetura com conjunto de instruções reduzido. Exemplos específicos serão vistos mais à frente neste capítulo. Começamos com uma discussão sobre motivações para arquitetura atuais com conjuntos de instruções complexas.

Figura 13.4 Abordagem de coloração de grafos





Por que CISC

Observamos a tendência para conjuntos de instruções mais ricos, o que inclui um número maior de instruções e instruções mais complexas. Dois motivos principais motivaram esta tendência: um desejo para simplificar compiladores e um desejo para melhorar o desempenho. Sob essas duas motivações estava a mudança para linguagens de programação de alto nível por parte dos programadores; projetistas tentaram projetar máquinas que oferecessem melhor suporte para linguagens de programação de alto nível.

Não é o intuito deste capítulo afirmar que os projetistas CISC tomaram a direção errada. Na verdade, como a tecnologia continua evoluindo e como as arquiteturas existem dentro de um espectro de categorias muito grande, uma avaliação preto no branco provavelmente nunca existirá. Por isso, os comentários que seguem têm apenas a intenção de identificar algumas falhas na abordagem CISC e de fornecer algum entendimento da motivação dos adeptos da abordagem RISC.

O primeiro motivo citado, a simplificação do compilador, parece óbvia. A tarefa do programador de compilador é gerar uma sequência de instruções de máquina para cada instrução de uma linguagem de alto nível. Se houver instruções de máquina que se assemelhem às instruções da linguagem de alto nível, esta tarefa será simplificada. Esta motivação foi disputada pelos pesquisadores RISC (HENNESSY et al., 1982^o; RADIN, 1983^o; PATTERSON e PIEPHO, 1982^o). Eles concluíram que instruções de máquina complexas são frequentemente difíceis de serem exploradas porque o compilador precisa encontrar os casos em que essa construção se encaixa perfeitamente. A tarefa de otimizar o código gerado para minimizar o tamanho do código, reduzir o total de execução de instruções e melhorar pipeline é muito mais difícil com um conjunto de instruções complexo. Uma prova disso são os estudos citados anteriormente neste capítulo que indicam que a maioria das instruções em um programa compilado são as relativamente simples.

Outro motivo importante citado é a expectativa de que um CISC produza programas menores e mais rápidos. Vamos analisar os dois aspectos desta declaração: que os programas serão menores e que executarão mais rapidamente.

Existem duas vantagens de programas menores. Primeira: como o programa ocupa menos memória, há uma economia nesse recurso. Com a memória hoje em dia ficando cada vez mais em conta, a vantagem potencial não é mais atraente. Mais importante ainda, programas menores devem melhorar o desempenho e isso irá acontecer de duas maneiras. Primeiro, menos instruções significam menos bytes de instruções para serem obtidos. Segundo, em um ambiente de paginação, programas menores ocupam menos páginas, reduzindo falhas de página.

O problema com esta linha de raciocínio é que está longe de ser certo que um programa CISC será menor que um programa RISC correspondente. Em muitos casos, o programa CISC expresso na linguagem de máquina simbólica pode ser *mais curto* (ou seja, menos instruções), porém o número de bits de memória ocupados pode não ser notavelmente *menor*. A Tabela 13.6 mostra os resultados de três estudos que compararam o tamanho de programas C compilados em uma variedade de máquinas, incluindo RISC I, a qual possui uma arquitetura de conjunto de instruções reduzido. Observe que há pouca ou nenhuma economia ao se usar um CISC no lugar de um RISC. É também interessante observar que o VAX, o qual tinha um conjunto de instruções muito mais complexo do que PDP-11, obtém muito pouca economia em relação a este último. Estes resultados foram confirmados pelos pesquisadores da IBM (Radin, 1983^o) que concluíram que IBM 801 (um RISC) produzia código que era 0,9 vez o tamanho do código de um IBM S/370. O estudo utilizou uma série de programas PL/I.

Tabela 13.6 Tamanho de código relativo a RISC I

	Patterson e Sequin (1982 ^a) 11 programas C	Katevenis (1983 ^a) 12 programas C	Heath (1984 ^a) 5 programas C
RISC I	1,0	1,0	1,0
VAX-11/780	0,8	0,67	
M68000	0,9		0,9
Z8002	1,2		1,12
PDP-11/70	0,9	0,71	

Há várias razões para esses resultados tão surpreendentes. Nós já observamos que os compiladores CISC tendem a favorecer instruções mais simples, de tal forma que a concisão de instruções complexas raramente vem em jogo. Além disso, como existem mais instruções em um CISC, *opcodes* maiores são necessários, produzindo instruções maiores. Finalmente, RISC tende a favorecer referências a registrador no lugar da memória, e o primeiro requer menos bits. Um exemplo deste último efeito será discutido em breve.

Então, a expectativa de que um CISC produza programas menores, com devidas vantagens, pode não se realizar. O segundo fator da motivação pelos conjuntos de instruções mais complexos era que a execução da instrução seria mais rápida. Parece fazer sentido que uma operação complexa da linguagem de alto nível vai executar mais rapidamente como sendo uma única instrução de máquina em vez de uma série de instruções mais primitivas. No entanto, por causa da tendência do uso dessas instruções mais simples, isso pode não ser verdade. A unidade de controle inteira deve ser feita de forma mais complexa e/ou o controle de armazenamento do microprograma deve ser maior para acomodar um conjunto de instruções mais rico. Cada fator desses aumenta o tempo de execução de instruções simples.

Na verdade, alguns pesquisadores concluíram que a aceleração da execução de funções complexas se deve não muito ao poder das instruções de máquina complexas, mas sim à sua permanência em memórias de controles de alta velocidade (Radin, 1983^o). Na realidade, a memória de controle de armazenamento age como uma cache de instruções. Assim, o projetista de hardware está em posição de tentar determinar que sub-rotinas ou funções serão usadas mais frequentemente e atribuí-las à memória do controle de armazenamento implementando-as no microcódigo. Os resultados foram menos que encorajadores. Nos sistemas S/390, instruções como Translate and Extended-Precision-Floating-Point-Divide residem em memórias de alta velocidade, enquanto a sequência envolvida em definir a chamada do procedimento ou iniciar ou tratar de interrupção estão na memória principal mais lenta.

Assim, não está nem um pouco claro que uma tendência pelo aumento na complexidade dos conjuntos de instruções seja apropriada. Isso levou vários grupos a seguirem o caminho oposto.



Características da arquitetura com conjuntos reduzidos de instruções

Embora várias abordagens para arquitetura com conjunto reduzido de instruções tenham sido implementadas, algumas características são comuns a todas elas:

- Uma instrução por ciclo.
- Operações registrador-para-registrador.
- Modos de endereçamento simples.
- Formatos de instruções simples.

Aqui faremos uma breve discussão sobre essas características. Exemplos específicos são explorados mais à frente neste capítulo.

A primeira característica relacionada é que há **uma instrução de máquina por ciclo de máquina**. Um *ciclo de máquina* é definido como o tempo necessário para obter dois operandos dos registradores, executar uma operação de ALU e armazenar o resultado em um registrador. Assim, as instruções de máquina RISC não deveriam ser mais complicadas do que as microinstruções em máquinas CISC (discutidas na Parte 4) e deveriam executar quase ao mesmo tempo. Com instruções simples de ciclo único, há pouca ou nenhuma necessidade de microcódigo; as instruções de máquina podem ser embutidas no hardware. Tais instruções deveriam executar mais rapidamente do que instruções de máquina comparáveis em outras máquinas, porque não é necessário acessar um microprograma de controle durante a execução da instrução.

A segunda característica é que a maioria das operações deve ser **registrador-para-registrador**, com apenas operações simples LOAD e STORE (CARREGAR e ARMAZENAR) acessando a memória. Este recurso de projeto simplifica o conjunto de instruções e, portanto, a unidade de controle também. Por exemplo, uma instrução RISC pode incluir apenas uma ou duas instruções ADD (por exemplo, adição inteira, adição com *carry*); o VAX possui 25 instruções ADD diferentes. Outro benefício é que uma arquitetura dessas encoraja a otimização do uso de registradores, para que os operandos frequentemente acessados permaneçam em armazenamentos de alta velocidade.

Esta ênfase em operações registrador-para-registrador é comum em projetos RISC. Máquinas CISC atuais fornecem tais instruções, mas também incluem operações memória-para-memória e operações mistas registrador/memória. Tentativas de comparar estas abordagens foram feitas em 1970, antes do surgimento do RISC. A Figura 13.5a

ilustra a abordagem. Arquiteturas hipotéticas evoluíram em tamanho de programas e número de bits de tráfego de memória. Resultados como este levaram um pesquisador a sugerir que futuras arquiteturas não deveriam conter registrador nenhum (MYERS, 1978¹). Alguém poderia se perguntar o que ele pensaria naquele tempo da máquina RISC Pyramid uma vez produzida e que tinha não menos do que 528 registradores.

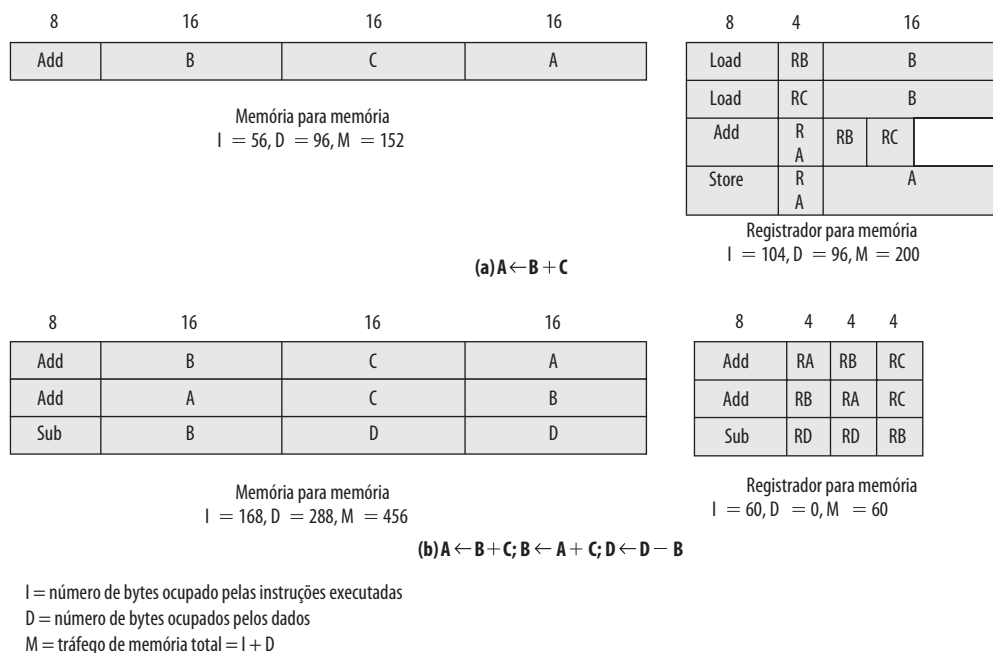
O que faltava nesses estudos era o reconhecimento do acesso frequente a um número pequeno de escalares locais e que, com um banco grande de registradores ou um compilador otimizado, a maioria dos operandos poderia ser mantida nos registradores por longos períodos. Assim, a Figura 13.5b poderia ser uma comparação mais justa.

A terceira característica é o uso de **modos de endereçamento simples**. Quase todas as instruções RISC usam endereçamento de registradores simples. Vários modos adicionais, tais como deslocamento relativo a PC, podem ser incluídos. Outros modos mais complexos podem ser sintetizados pelo software a partir dos mais simples. Novamente, este recurso de projeto simplifica o conjunto de instruções e a unidade de controle.

A característica comum final é o uso de **formatos de instruções simples**. Geralmente, apenas um ou alguns poucos formatos são usados. O tamanho da instrução é fixo e ajustado dentro do limite da palavra. A posição de campos, especialmente o *opcode*, é fixa. Este recurso tem uma série de benefícios. Com campos fixos, decodificação de *opcode* e o acesso a registradores de operandos podem ocorrer ao mesmo tempo. Formatos simples simplificam a unidade de controle. A leitura de instruções é otimizada porque são obtidas unidades do tamanho da palavra são obtidas. Esse ajuste a limites da palavra significa também que uma única instrução não ultrapassa os limites da página.

Consideradas juntas, estas características podem ser avaliadas para determinar os benefícios potenciais de desempenho da abordagem RISC. Certa quantidade de “evidência circunstancial” pode ser demonstrada. Primeiro, compiladores com otimização mais eficientes podem ser desenvolvidos. Com instruções mais primitivas, existem mais oportunidades para mover funções fora dos laços de repetição, reorganizar o código visando à eficiência, maximizando o uso de registradores e assim por diante. É até possível computar partes das instruções complexas em tempo de compilação. Por exemplo, a instrução Mover Caracteres (MVC) do S/390 move uma cadeia de caracteres de uma posição para outra. Cada vez que ela é executada, o movimento vai depender do tamanho da cadeia, se e em qual direção as posições se sobrepõem e quais são as características do alinhamento. Na maioria dos casos, todas essas situações serão conhecidas em tempo de compilação. Assim, o compilador poderia produzir uma sequência otimizada de instruções primitivas para essa função.

Figura 13.5 Duas comparações de abordagens registrador-para-registrador e memória-para-memória



Um segundo ponto já observado é que a maioria de instruções geradas por um compilador são relativamente simples. Parece razoável que uma unidade de controle construída especialmente para essas instruções com uso de pouco ou nenhum microcódigo possa executá-las mais rapidamente do que um CISC comparável.

Um terceiro ponto tem a ver com o uso de pipeline de instruções. Os pesquisadores RISC acharam que a técnica de pipeline de instruções pode ser aplicada muito mais eficientemente com um conjunto de instruções reduzido. Nós analisamos este ponto em mais detalhes daqui a pouco.

Um último ponto, e de certa forma menos significativo, indica que os processadores RISC respondem melhor às interrupções porque estas são verificadas entre operações elementares. As arquiteturas com instruções complexas ou restringem interrupções aos limites da instrução ou precisam definir pontos de interrupção especiais e implementar mecanismos para reiniciar a instrução.

As vantagens para um desempenho melhorado estão do lado da arquitetura de conjunto reduzido de instruções, mas talvez alguém ainda poderia ter um argumento para o CISC. Um número de estudos tem sido feito, mas não em máquinas de tecnologia e potência comparáveis. Além disso, a maioria desses estudos não tentou separar os efeitos de um conjunto reduzido de instruções e os efeitos de um banco grande de registradores. “A prova circunstancial”, no entanto, é sugestiva.



Características CISC versus RISC

Depois do entusiasmo inicial pelas máquinas RISC, houve um entendimento crescente de que: (1) o projeto RISC pode se beneficiar da inclusão de alguns recursos CISC e (2) o projeto CISC pode se beneficiar da inclusão de alguns recursos RISC. O resultado é que os projetos RISC mais recentes, notavelmente o PowerPC, não são mais “puramente” RISC, e os projetos CISC mais recentes, notavelmente Pentium II e últimos modelos do Pentium, incorporam de fato algumas características RISC.

Uma comparação interessante em Mashey (1995⁵) fornece algum esclarecimento sobre essa questão. A Tabela 13.7 mostra uma série de processadores e os compara por uma série de características. Para os propósitos desta comparação, os seguintes itens são considerados típicos de um RISC clássico:

1. Um tamanho único de instrução.
2. O tamanho é normalmente de 4 bytes.
3. Um número menor de modos de endereçamento, normalmente menos de cinco. Este parâmetro é difícil de determinar. Na tabela, modos literais e de registradores não são contados e formatos diferentes com tamanhos de offset diferentes são contados separadamente.
4. Nenhum endereçamento indireto que requer um acesso à memória para obter o endereço de um operando na memória.
5. Nenhuma operação que combina leitura/escrita com aritmética (por exemplo, adicionar da memória, adicionar para memória).
6. Não mais do que um operando endereçado em memória por instrução.
7. Não suporta alinhamento arbitrário de dados para operações de leitura/escrita.
8. Número máximo de usos da unidade de gerenciamento de memória (MMU) para um endereço de dados em uma instrução.
9. Número de bits para especificadores registradores inteiros igual a cinco ou mais. Isto significa que ao menos 32 registradores inteiros podem ser explicitamente referenciados em um momento.
10. Número de bits para especificadores de registradores de ponto flutuante igual a quatro ou mais. Isto significa que ao menos 16 registradores de ponto flutuante podem ser referenciados explicitamente em um momento.

Os itens de 1 a 3 são uma indicação da complexidade de decodificação da instrução. Os itens 4 a 8 sugerem a facilidade ou dificuldade de pipeline, especialmente na presença de requisitos da memória virtual. Os itens 9 e 10 são relacionados à habilidade para obter uma boa vantagem dos compiladores.

Os primeiros oito processadores da tabela são claramente arquiteturas RISC, os próximos 5 são claramente CISC e os dois últimos são processadores frequentemente pensados como RISC que possuem muitas características CISC.

Tabela 13.7 Características de alguns processadores

Processador	Número de tamanhos de instrução	Tamanho máximo da instrução em bytes	Número de modos de endereçamento	Endereçamento indireto	Leitura/escrita combinada com aritmética	Número máximo de operandos de memória	Endereçamento não alinhado permitido	Número máximo de usos da MMU	Número de bits para especificadores de registradores inteiros	Número de bits para especificadores de registradores PF
AMD29000	1	4	1	não	não	1	não	1	8	3 ^a
MIPS R2000	1	4	1	não	não	1	não	1	5	4
SPARC	1	4	2	não	não	1	não	1	5	4
MC88000	1	4	3	não	não	1	não	1	5	4
HP PA	1	4	10 ^a	não	não	1	não	1	5	4
IBM RT/PC	2 ^a	4	1	não	não	1	não	1	4 ^a	3 ^a
IBM RS/6000	1	4	4	não	não	1	sim	1	5	5
Intel i860	1	4	4	não	não	1	não	1	5	4
IBM 3090	4	8	2 ^b	não ^b	sim	2	sim	4	4	2
Intel 80486	12	12	15	não ^b	sim	2	sim	4	3	3
NSC 32016	21	21	23	sim	sim	2	sim	4	3	3
MC68040	11	22	44	sim	sim	2	sim	8	4	3
VAX	56	56	22	sim	sim	6	sim	24	4	0
Clipper	4 ^a	8 ^a	9 ^a	não	não	1	0	2	4 ^a	3 ^a
Intel 80960	2 ^a	8 ^a	9 ^a	não	não	1	sim ^a	—	5	3 ^a

^a RISC que não está conforme a esta característica.

^b CISC que não está conforme a esta característica.



13.5 Pipeline no RISC



Pipeline com instruções regulares

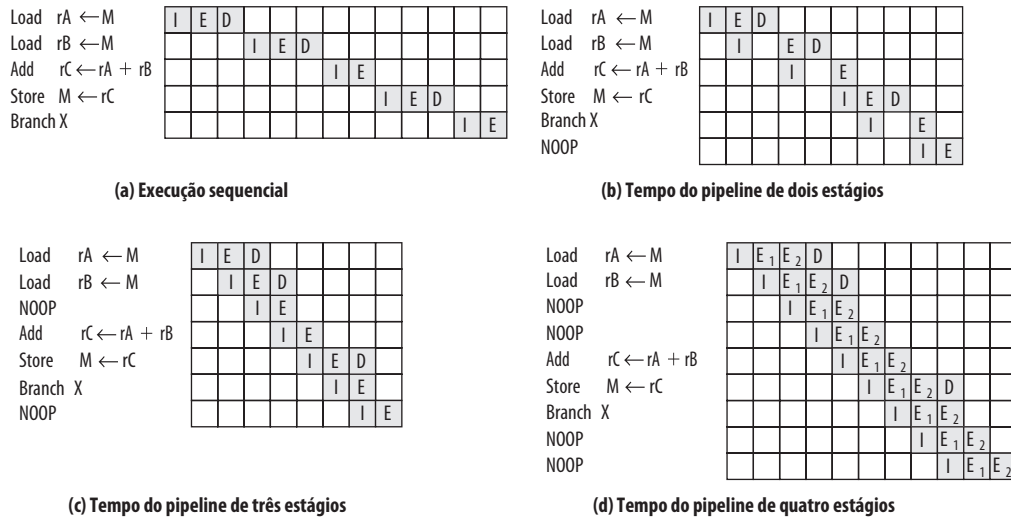
Conforme discutimos na Seção 12.4, o pipeline de instruções é frequentemente usado para melhorar o desempenho. Vamos reconsiderar isso dentro do contexto de uma arquitetura RISC. A maioria das instruções é do tipo registrador-para-registrador e um ciclo da instruções possui os dois estágios a seguir:

- I: Busca da instrução.
- E: Execução. Efetua uma operação de ALU com saída e entrada de registradores.

Para operações de carregar e armazenar, três estágios são requeridos:

- I: Busca da instrução.
- E: Execução. Calcular endereço de memória.
- D: Memória. Operação registrador-para-memória ou memória-para-registrador.

A Figura 13.6a ilustra o tempo de uma sequência de instruções sem pipeline. Claramente, este é um processo de muito desperdício. Até um pipeline muito simples pode substancialmente melhorar o desempenho. A Figura 13.6b mostra um esquema de pipeline de dois estágios, onde os estágios I e E de duas instruções diferentes são efetuados simultaneamente. Os dois estágios do pipeline são um de busca da instrução e um estágio de execução/memória que executa uma instrução, incluindo operações registrador-para-memória e memória-para-registrador. Desta forma

Figura 13.6 Os efeitos de pipeline

podemos observar que o estágio de busca de instrução da segunda instrução pode ser executado em paralelo com a primeira parte do estágio execução/memória. No entanto, o estágio execução/memória da segunda instrução deve ser atrasado até que a primeira instrução esvazie o segundo estágio do pipeline. Este esquema pode aumentar em até duas vezes a taxa de execução de um esquema em série. Dois problemas impedem que o aumento máximo da velocidade seja alcançada. Primeiro, assumimos que uma memória de acesso único é usada e que apenas um acesso à memória é possível por estágio. Isso requer a adição de um estágio de espera em algumas instruções. Segundo, uma instrução de desvio interrompe o fluxo sequencial da execução. Para acomodar isso com o uso menor de circuitos, uma instrução NOOP pode ser inserida no fluxo das instruções pelo compilador ou assembler.

O pipeline pode ainda ser melhorado permitindo dois acessos à memória por estágio. Isso produz a sequência mostrada na Figura 13.6c. Agora, até três instruções podem ser sobrepostas e o melhoramento é de fator de 3. Novamente, instrução de desvio faz com que a aceleração não atinja o máximo possível. Além disso, observe que as dependências de dados são afetadas. Se uma instrução precisa de um operando que é alterado por uma instrução anterior, um atraso é necessário. Novamente, isso pode ser conseguido com um NOOP.

O pipeline discutido até agora funciona melhor se os três estágios forem de duração aproximadamente igual. Como o estágio E normalmente envolve uma operação ALU, ele pode ser mais demorado. Neste caso, podemos dividi-lo em dois subestágios:

- E₁: leitura do banco de registradores.
- E₂: operação da ALU e escrita em registrador.

Por causa da simplicidade e regularidade de um conjunto de instruções RISC, o projeto de três ou quatro estágios é facilmente alcançado. A Figura 13.6d mostra o resultado com um pipeline de quatro estágios. Até quatro instruções podem estar em curso ao mesmo tempo e o potencial máximo do aumento da velocidade é um fator de 4. Observe novamente o uso de NOOPs por causa dos atrasos de desvios e dados.



Otimização de pipeline

Por causa da natureza simples e regular das instruções RISC, os esquemas de pipeline podem ser eficientemente empregados. Há poucas variações na duração da execução de instruções e o pipeline pode ser elaborado para refletir isso. No entanto, vimos que as dependências de dados e os desvios reduzem a taxa de execução total.

DESVIO ATRASADO Para compensar essas dependências, técnicas de reorganização de código foram desenvolvidas. Primeiro, vamos considerar as instruções de desvio. *Desvio atrasado*, uma maneira de aumentar a eficiência do pipeline, faz uso de um desvio que não toma efeito até depois da execução da instrução seguinte (por isso o tempo

atrasado). A posição da instrução imediatamente depois do desvio é referida como *delay slot*. Este procedimento estranho é ilustrado na Tabela 13.8. Na coluna chamada de “desvio normal” podemos ver uma instrução normal em linguagem de máquina.

Depois que a linha 102 é executada, a próxima instrução a ser executada é 105. Para regularizar o pipeline, um NOOP é inserido depois desse desvio. No entanto, um melhor desempenho é alcançado se as instruções nas linhas 101 e 102 forem trocadas.

A Figura 13.7 mostra o resultado. A Figura 13.7a mostra a abordagem tradicional de pipeline, do tipo que foi discutido no Capítulo 12 (para um exemplo veja as Figuras 12.11 e 12.12).

A instrução JUMP é lida no tempo 3. No tempo 4, ela é executada ao mesmo tempo que a instrução 103 (instrução ADD) é lida. Como ocorre um JUMP, o qual atualiza o contador de programas, o pipeline precisa ser esvaziado com relação à instrução 103; no tempo 5, a instrução 105, a qual é o alvo de JUMP, é carregada. A Figura 13.7b mostra o mesmo pipeline tratado por uma organização RISC típica. O tempo é o mesmo. No entanto, por causa da inserção da instrução NOOP, nós não precisamos de circuitos para esvaziar o pipeline; o NOOP simplesmente é executado sem nenhum efeito. A Figura 13.7c mostra o uso do desvio atrasado. A instrução JUMP é lida no tempo 2, antes da instrução ADD, que é obtida no tempo 3. Observe, no entanto, que a instrução ADD é lida antes da execução da instrução JUMP ter a chance de alterar o contador de programa. Por isso, durante o tempo 4, a instrução ADD é executada ao mesmo tempo que a instrução 105 é lida. Assim, as semânticas originais do programa são mantidas, mas um ciclo de clock a menos é necessário para execução.

Esta troca de instruções funciona com sucesso para desvios incondicionais, chamadas e retornos. Para desvios condicionais, este procedimento não pode ser aplicado cegamente. Se a condição testada para o desvio pode ser alterada pela instrução imediatamente anterior, então o compilador não deve fazer a troca, mas sim inserir uma instrução NOOP. Caso contrário, o compilador pode procurar inserir uma instrução útil depois do desvio. A experiência com os sistemas RISC de Berkeley e o IBM 801 é que a maioria das instruções de desvio condicional pode ser otimizada dessa maneira (Patterson e Sequin, 1982^a; Radin, 1983^o).

LEITURA ATRASADA Uma tática semelhante, chamada leitura atrasada (*delay load*), pode ser usada em instruções LOAD. Em instruções LOAD, o registrador que é o alvo da leitura é bloqueado pelo processador. O processador então continua a execução do fluxo de instruções até que alcance uma instrução que precisa desse registrador, ponto no qual ele fica ocioso até que a leitura esteja completada. Se o compilador puder rearranjar as instruções para que o trabalho útil possa ser feito enquanto ocorre leitura dentro do pipeline, a eficiência será aumentada.

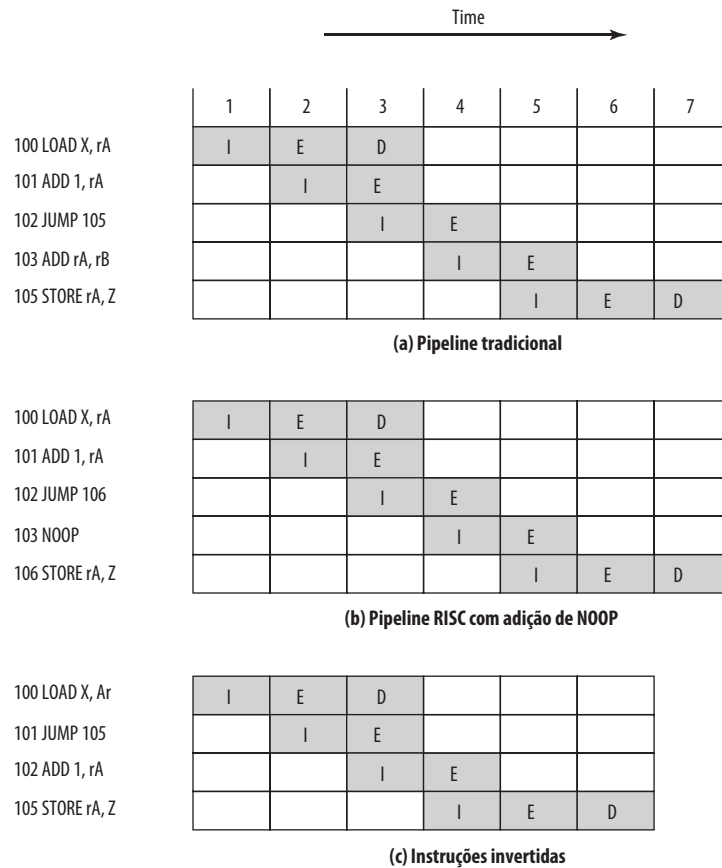


Simulador de laço desenrolado

LAÇO DESENROLADO Outra técnica de compiladores para melhorar o paralelismo de instruções é o laço desenrolado (BACON, GRAHAM e SHARP, 1994^o). Desenrolar replica, algumas vezes, o corpo de um laço em um número de vezes chamado de fator de desenrolar (u), e faz a iteração pelo passo u em vez de passo 1.

Tabela 13.8 Desvio normal e atrasado

Endereço	Desvio normal	Desvio atrasado	Desvio atrasado otimizado
100	LOAD X, rA	LOAD X, rA	LOAD X, rA
101	ADD 1, rA	ADD 1, rA	JUMP 105
102	JUMP 105	JUMP 106	ADD 1, rA
103	ADD rA, rB	NOOP	ADD rA, rB
104	SUB rC, rB	ADD rA, rB	SUB rC, rB
105	STORE rA, Z	SUB rC, rB	STORE rA, Z
106		STORE rA, Z	

Figura 13.7 Uso de desvio atrasado

Desenrolar pode melhorar o desempenho:

- Reduzindo a sobrecarga de laço.
- Aumentando o paralelismo de instruções, melhorando o desempenho do pipeline.
- Melhorando a localidade de registradores, cache de dados ou TLB.

A Figura 13.8 ilustra todas essas três melhorias em um único exemplo. A sobrecarga de laço é cortada pela metade porque duas iterações são executadas antes do teste e o desvio é feito no topo do laço. Cresce o paralelismo de instruções porque a segunda atribuição pode ser efetuada enquanto os resultados da primeira estão sendo armazenados, e as variáveis do laço sendo atualizadas. Se os elementos do vetor são atribuídos aos registradores, o posicionamento dos registradores vai melhorar porque $a[i]$ e $a[i + 1]$ são usados duas vezes no corpo do laço, reduzindo o número de leituras por iteração de três para dois.

Como uma observação final, podemos destacar que o projeto do pipeline de instruções não deve ser usado de maneira isolada de outras técnicas de otimização aplicadas ao sistema. Por exemplo, Bradlee, Eggers e Henry (1991^u) mostram que o escalonamento de instruções para o pipeline e a alocação dinâmica de registradores devem ser considerados juntos para alcançar a melhor eficiência.



13.6 MIPS R4000

Um dos primeiros chips RISC disponível comercialmente foi desenvolvido por MIPS Technology Inc. O sistema foi inspirado por um sistema experimental, que também usava o nome MIPS, desenvolvido em Standford (HENNESSY, 1984^v). Nesta seção, analisamos o MIPS R4000. Ele possui substancialmente a mesma arquitetura e o mesmo

Figura 13.8 Laço desenrolado

```
do i=2, n-1
    a[i] = a[i] + a[i-1] * a[i+1]
end do
```

(a) Laço original

```
do i=2, n-2, 2
    a[i] = a[i] + a[i-1] * a[i+1]
    a[i+1] = a[i+1] + a[i] * a[i+2]
end do

if (mod(n-2, 2) = 1) then
    a[n-1] = a[n-1] + a[n-2] * a[n]
end if
```

(b) Laço desenrolado duas vezes

conjunto de instruções de projetos MIPS anteriores: R2000 e R3000. A diferença mais significativa é que o R4000 usa 64 em vez de 32 bits para os caminhos de dados internos e externos, endereços, registradores e ALU.

O uso de 64 bits possui uma série de vantagens em relação a uma arquitetura de 32 bits. Ele permite um espaço de endereçamento maior — grande o suficiente para um sistema operacional mapear mais do que um terabyte de arquivos diretamente para a memória virtual facilitando o acesso. Com 1 terabyte e discos maiores, comuns hoje em dia, o espaço de endereçamento de 4 GB de uma máquina de 32 bits torna-se limitado. Além disso, a capacidade de 64 bits permite ao R4000 processar dados como números de ponto flutuante de precisão dupla e cadeias de caracteres de até oito caracteres em uma única ação.

O chip do processador R4000 é particionado em duas seções, uma contendo CPU e outra contendo um coprocessador para gerenciamento de memória. O processador possui uma arquitetura muito simples. A intenção foi projetar um sistema no qual a lógica de execução de instruções fosse a mais simples possível, deixando espaço disponível para lógica de melhoria de desempenho (por exemplo, a unidade de gerenciamento de memória inteira).

O processador suporta 32 registradores de 64 bits. Ele também possui uma cache de alta velocidade de 128 KB, sendo metade para instruções e metade para dados. Cache relativamente grande (IBM 3090 fornece cache de 128 a 256 KB) possibilita que o sistema guarde grandes conjuntos de códigos de programa e de dados locais para o processador, desocupando o barramento da memória principal e evitando a necessidade de um banco grande de registradores pela lógica de janelas.



Conjunto de instruções

A Tabela 13.9 mostra o conjunto de instruções básico para todos os processadores da série MIPS R. Todas as instruções do processador são codificadas em um formato de uma única palavra de 32 bits. Todas as operações de dados são de registrador para registrador; as únicas referências de memória são operação puramente de leitura/escrita.

O R4000 não faz uso de códigos condicionais. Se uma instrução gera uma condição, os flags correspondentes são armazenadas em um registrador de uso geral. Isso evita a necessidade, de uma lógica especial, para lidar com códigos condicionais porque eles afetam o mecanismo de pipeline e a reordenação de instruções pelo compilador. Em vez disso, os mecanismos já implementados para lidar com dependências de valores de registradores são empregados. Além disso, as condições mapeadas para bancos de registradores são sujeitas às mesmas otimizações em tempo de compilação de alocações e reúso como outros valores armazenados nos registradores.

Como acontece com a maioria das máquinas RISC, o MIPS usa um tamanho único da instrução de 32 bits. Este tamanho único da instrução simplifica a leitura e a decodificação da instrução e simplifica também a interação da

Tabela 13.9 Conjunto de instruções da série R de MIPS

OP	Descrição	OP	Descrição
	Instruções de Carregar/Armazenar	SLLV	Deslocamento à esquerda lógico variável
LB	Carregar byte	SRLV	Deslocamento à direita lógico variável
LBU	Carregar byte sem sinal	SRAV	Deslocamento à direita aritmético variável
LH	Carregar metade da palavra		Instruções de multiplicação/divisão
LHU	Carregar metade da palavra sem sinal	MULT	Multiplicar
LW	Carregar palavra	MULTU	Multiplicar sem sinal
LWL	Carregar palavra da esquerda	DIV	Dividir
LWR	Carregar palavra da direita	DIVU	Dividir sem sinal
SB	Armazenar byte	MFHI	Mover de HI
SH	Armazenar metade da palavra	MTHI	Mover para HI
SW	Armazenar palavra	MFLO	Mover de LO
SWL	Armazenar palavra da esquerda	MTLO	Mover para LO
SWR	Armazenar palavra da direita		Instruções de salto e desvio
	Instruções aritméticas (ALU, com operando imediato)	J	Saltar
ADDI	Adicionar imediato	JAL	Saltar com ligação
ADDIU	Adicionar imediato sem sinal	JR	Saltar com registrador
SLTI	Atribuir 1 se menor que imediato	JALR	Saltar e ligar com registrador
SLTIU	Atribuir 1 se menor que imediato sem sinal	BEQ	Desviar quando igual
ANDI	AND imediato	BNE	Desviar quando não igual
ORI	OR imediato	BLEZ	Desviar quando menor ou igual a zero
XORI	Exclusive-OR imediato	BGTZ	Desviar quando maior que zero
LUI	Carregar metade superior com o imediato	BLTZ	Desviar quando menor que zero
	Instruções aritméticas (operando 3, tipo R)	BGEZ	Desviar quando maior ou igual a zero
ADD	Adicionar	BLTZAL	Desviar quando menor que zero com ligação
ADDU	Adicionar sem sinal	BGEZAL	Desviar quando maior ou igual a zero com ligação
SUB	Subtrair		Instruções do coprocessador
SUBU	Subtrair sem sinal	LWCz	Carregar palavra no coprocessador
SLT	Definir em menos que	SWCz	Armazenar palavra do coprocessador
SLTU	Definir em menos que sem sinal	MTCz	Mover para coprocessador
AND	AND	MFCz	Mover do coprocessador
OR	OR	CTCz	Mover controle para coprocessador
XOR	Exclusive-DR	CFCz	Mover controle do coprocessador
NOR	NOR	COPz	Operação do coprocessador
	Instruções de deslocamento	BCzT	Desviar quando coprocessador z verdadeiro
SLL	Deslocamento à esquerda lógico	BCzF	Desviar quando coprocessador z falso
SRL	Deslocamento à direita lógico		Instruções especiais
SRA	Deslocamento à direita aritmético	SYSALL	Chamada de sistema
		Break	Parada

leitura de instrução com a unidade de gerenciamento da memória virtual (isto é, as instruções não ultrapassam os limites da palavra ou da página). Os três formatos de instrução (Figura 13.9) compartilham formatação comum de *opcodes* e referências a registradores, simplificando a decodificação das instruções. O efeito das instruções mais complexas pode ser sintetizado em tempo de compilação.

Apenas o modo de endereçamento de memória mais simples e mais frequentemente usado é implementado no hardware. Todas as referências de memória consistem de um offset de 16 bits de um registrador de 32 bits. Por exemplo, a instrução “carregar palavra” (*load word* — *lw*) tem o formato de

```
lw r2, 128(r3) /* carrega palavra no registrador 2 do offse 128 a partir do endereço armazenado no registrador 3*/
```

Cada um dos 32 registradores de uso geral pode ser usado como registrador base. Um registrador, *r0*, sempre contém 0.

O compilador faz uso de múltiplas instruções de máquina para sintetizar modos de endereçamento típicos em máquinas convencionais. Aqui está um exemplo de Chow et al. (1987^w), o qual usa a instrução LUI (carregar superior imediato na parte superior — *load upper immediate*). Esta instrução carrega a metade superior de um registrador com um valor imediato de 16 bits, definindo a metade inferior igual zero. Considere uma instrução da linguagem de montagem que usa um argumento imediato de 32 bits

```
lw r2, #imm(r4) /* carrega palavra no registrador usando um offset imediato #imm de 32 bits
/* offset do endereço no registrador 4 para registrador 2
```

Esta instrução pode ser compilada para as seguintes instruções MIPS

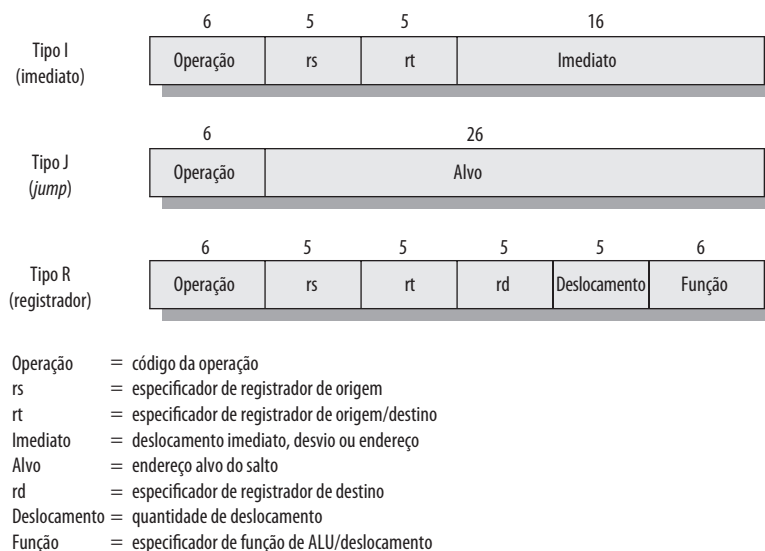
```
lui r1, #imm-hi /* onde #imm-hi são os 16 bits de ordem mais alta de #imm
addu r1, r1, r4 /* adiciona #imm-hi sem sinal para r4 e coloca em r1
lw r2, #imm-lo(r1) /* onde #imm-lo são 16 bits de ordem mais baixa de #imm
```



Pipeline de instruções

Com a sua arquitetura simplificada de instruções, o MIPS pode alcançar uma eficiência grande em seu pipeline. É instrutivo analisar a evolução do pipeline do MIPS, porque ela ilustra a evolução do pipeline no RISC de um modo geral.

Figura 13.9 Formatos das instruções MIPS



Os primeiros sistemas RISC experimentais e a primeira geração de processadores RISC comerciais alcançam velocidades de execução que se aproximam de uma instrução por um ciclo de clock do sistema. Para melhorar esse desempenho, duas classes de processadores evoluíram para oferecer a execução de múltiplas instruções por ciclo de clock: arquiteturas superescalares e superpipeline. Uma arquitetura superescalares basicamente replica cada um dos estágios do pipeline para que duas ou mais instruções no mesmo estágio do pipeline possam ser processadas simultaneamente. Uma arquitetura superpipeline é aquela que usa estágios do pipeline cada vez mais minuciosos. Com mais estágios, mais instruções podem estar no pipeline ao mesmo tempo, aumentando o paralelismo.

Ambas as abordagens possuem limitações. Com pipeline superescalar, as dependências entre as instruções em diferentes pipelines podem tornar o sistema mais lento. Além disso, um aumento de lógica é necessário para coordenar essas dependências. Com superpipeline, existe um *overhead* associado com a transferência de instruções de um estágio para outro.

O Capítulo 14 é dedicado a um estudo de arquiteturas superescalares. O MIPS R4000 é um bom exemplo de uma arquitetura RISC com superpipeline.

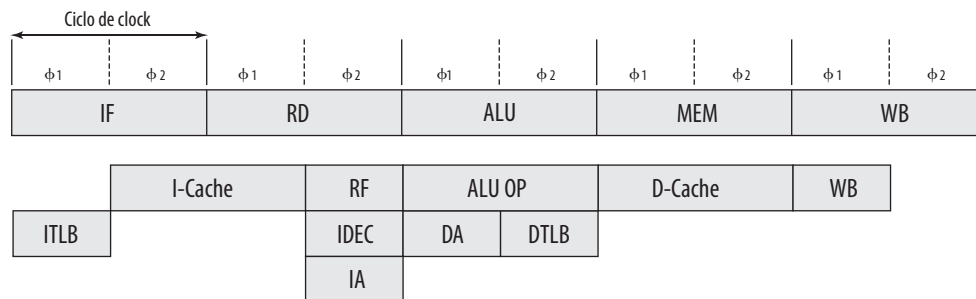


Simulador de pipeline de cinco estágios do MIPS R3000

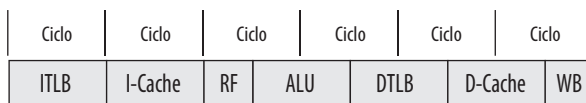
A Figura 13.10a mostra o pipeline de instruções de R3000. No R3000, o pipeline avança uma vez por ciclo de clock. O compilador MIPS é capaz de reordenar instruções para preencher os *delay slots* com código de 70 a 90% do tempo. Todas as instruções seguem a mesma sequência dos cinco estágios do pipeline:

- Busca de instrução.
- Busca operando de origem no arquivo de registradores

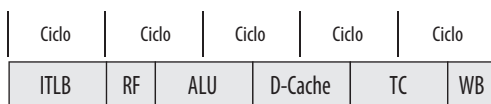
Figura 13.10 Melhorando pipeline do R3000



(a) Pipeline do R3000 detalhado



(b) Pipeline do R3000 modificado com latências reduzidas



(c) Pipeline do R3000 otimizado com acesso paralelo a TLB e cache

- IF = busca da instrução
- RD = leitura
- MEM = acesso à memória
- WB = atualizar
- I-Cache = acesso à cache de instruções
- RF = busca do operando do registrador
- D-Cache = acesso à cache de dados
- ITLB = tradução endereço da instrução
- IDEC = decodificação da instrução
- IA = calcular endereço da instrução
- DA = calcular endereço virtual de dados
- DTLB = traduzir endereço de dados
- TC = verificar rótulo de cache de dados

- Operação de ALU ou geração de endereço de operando de dados.
- Referência a dados de memória.
- Atualizar o banco de registradores.

Conforme ilustrado na Figura 13.10a, não há apenas um paralelismo por causa do pipeline mas também um paralelismo dentro da execução de uma única instrução. Um ciclo de clock de 60 ns é dividido em dois estágios de 30 ns. Instruções externas e operações de acesso a dados da cache requerem, cada uma, 60 ns, assim como as principais operações internas (OP, DA, IA). A decodificação de uma instrução é uma operação mais simples, reque-rendo apenas um único estágio de 30 ns, sobreposta com leitura de registrador na mesma instrução. Cálculo de um endereço para uma instrução de desvio também sobrepõe a decodificação da instrução e leitura do registrador, de tal forma que um desvio na instrução i pode endereçar o acesso ICACHE da instrução $i + 2$. De maneira semelhante, uma leitura na instrução i obtém os dados que são usados imediatamente por OP da instrução $i + 1$, enquanto um resultado de ALU/deslocamento passa diretamente pela instrução $i + 1$ sem nenhum atraso. Este acoplamento forte entre instruções torna o pipeline altamente eficiente.

Analisando em mais detalhes, cada ciclo de clock é dividido em estágios separados denotados como $\phi 1$ e $\phi 2$. As funções efetuadas em cada estágio são resumidas na Tabela 13.10.

O R4000 incorpora uma série de avanços técnicos em relação ao R3000. O uso de tecnologia mais avançada permite que o tempo do ciclo de clock seja diminuído pela metade, para 30 ns, e que o tempo de acesso ao arquivo de registradores seja diminuídos pela metade. Além disso, há uma densidade maior no chip, o que possibilita que o cache de instruções e a de dados sejam incorporados nele. Antes de analisar o pipeline final do R4000, vamos considerar como o pipeline do R3000 pode ser modificado para melhorar o desempenho usando a tecnologia do R4000.

A Figura 13.10b mostra o primeiro passo. Lembre que os ciclos desta figura são metade dos da Figura 13.10a. Como não estão no mesmo chip, os estágios de cache de instrução e de dados demoram apenas metade do tempo; então eles ainda ocupam apenas um ciclo de clock. Novamente, por causa do aumento da velocidade de acesso ao banco de registradores, leituras e escritas de registradores ainda ocupam apenas metade de um ciclo de clock.

Como as caches do R4000 estão no chip, a tradução do endereço virtual para físico pode atrasar o acesso à cache. Este atraso é reduzido com a implementação de caches indexadas virtualmente e usando o acesso à cache e tradução de endereços em paralelo. A Figura 13.10c mostra o pipeline otimizado do R3000 com suas melhorias. Por causa da compressão de eventos, a verificação do rótulo (*tag*) da cache de dados é feita separadamente no próximo ciclo depois do acesso à cache. Esta verificação determina se o item de dados está na cache.

Tabela 13.10 Estágios do pipeline do R3000

Estágio do pipeline	Fase	Função
IF	$\phi 1$	Usando a TLB, traduz um endereço virtual da instrução para um endereço físico (depois da decisão do desvio)
IF	$\phi 2$	Envia o endereço físico para endereço da instrução
RD	$\phi 1$	Retorna instrução da cache de instruções Compara rótulos e validade da instrução lida
RD	$\phi 2$	Decodifica instrução Lê banco de registradores Se for desvio, calcula endereço do alvo do desvio
ALU	$\phi 1 + \phi 2$	Se for uma operação registrador para registrador, operação aritmética ou lógica é executada
ALU	$\phi 1$	Se for um desvio, decide se o desvio deve ou não ser tomado Se for uma referência de memória (carregar ou armazenar), calcula o endereço virtual dos dados
ALU	$\phi 2$	Se for uma referência de memória, traduz endereço virtual dos dados para endereço físico usando TLB
MEM	$\phi 1$	Se for uma referência de memória, envia endereço físico para cache
MEM	$\phi 2$	Se for uma referência de memória, retorna os dados da cache de dados e verifica as marcações
WB	$\phi 1$	Escreve no banco de registradores

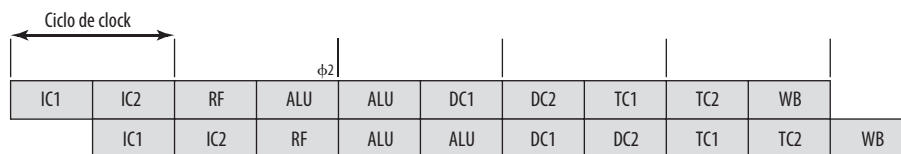
Em um sistema com superpipeline, o hardware existente é usado várias vezes por ciclo inserindo registradores de pipeline para dividir cada estágio. Essencialmente, cada estágio de superpipeline opera em um múltiplo da frequência base do clock, onde o múltiplo depende do grau do superpipeline. A tecnologia do R4000 tem a velocidade e a densidade para permitir superpipeline de grau 2. A Figura 13.11a mostra o pipeline do R3000 otimizado usando este superpipeline. Observe que isto é essencialmente a mesma estrutura dinâmica da Figura 13.10c.

Outras melhorias podem ser feitas. Para o R4000, foi projetado um somador maior e mais especializado. Isso torna possível executar operações da ALU com o dobro da velocidade. Outras melhorias permitem a execução de leituras e gravações com o dobro da velocidade. O pipeline resultante é mostrado na Figura 13.11b.

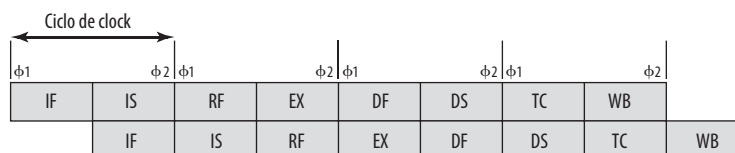
O R4000 possui oito estágios de pipeline, o que significa que no máximo oito instruções podem estar no pipeline ao mesmo tempo. O pipeline avança numa taxa de dois estágios por ciclo de clock. Os oito estágios do pipeline são:

- **Busca da instrução primeira metade:** endereço virtual é apresentado à cache de instruções e à TLB.
- **Busca da instrução segunda metade:** cache de instrução retorna a instrução e TLB gera endereço físico.
- **Banco de registradores:** três atividades ocorrem em paralelo:
 - Instrução é decodificada e a verificação de condições de bloqueios é feita (ou seja, esta instrução depende do resultado de uma instrução anterior).
 - Verificação do rótulo de cache de instruções é feita.
 - Operandos são obtidos do banco de registradores.
- **Executar instrução:** uma das três atividades pode ocorrer:
 - Se a instrução é uma operação registrador-para-registrador, ALU executa a operação aritmética ou lógica.
 - Se a instrução é leitura ou escrita, o endereço virtual de dados é calculado.
 - Se a instrução é um desvio, o endereço virtual do alvo do desvio é calculado e as condições de desvio são verificadas.
- **Primeira cache de dados:** endereço virtual é apresentado à cache de dados e à TLB.
- **Segunda cache de dados:** TLB gera o endereço físico e a cache de dados retorna a instrução.
- **Verificação de rótulos:** marcações de cache são verificadas para leitura e escrita.
- **Atualização:** o banco de registradores é atualizado com o resultado da instrução.

Figura 13.11 Superpipeline teórico do R3000 e superpipeline real do R4000



(a) Implementação de superpipeline do pipeline otimizado do R3000



(b) Pipeline do R4000

IF = busca de instrução primeira metade
 IS = busca de instrução segunda metade
 RF = busca do operandos do registrador
 EX = executar instrução
 IC = cache de instrução

DC = cache de dados
 DF = cache de dados primeira metade
 DS = cache de dados segunda metade
 TC = verificação de rótulos

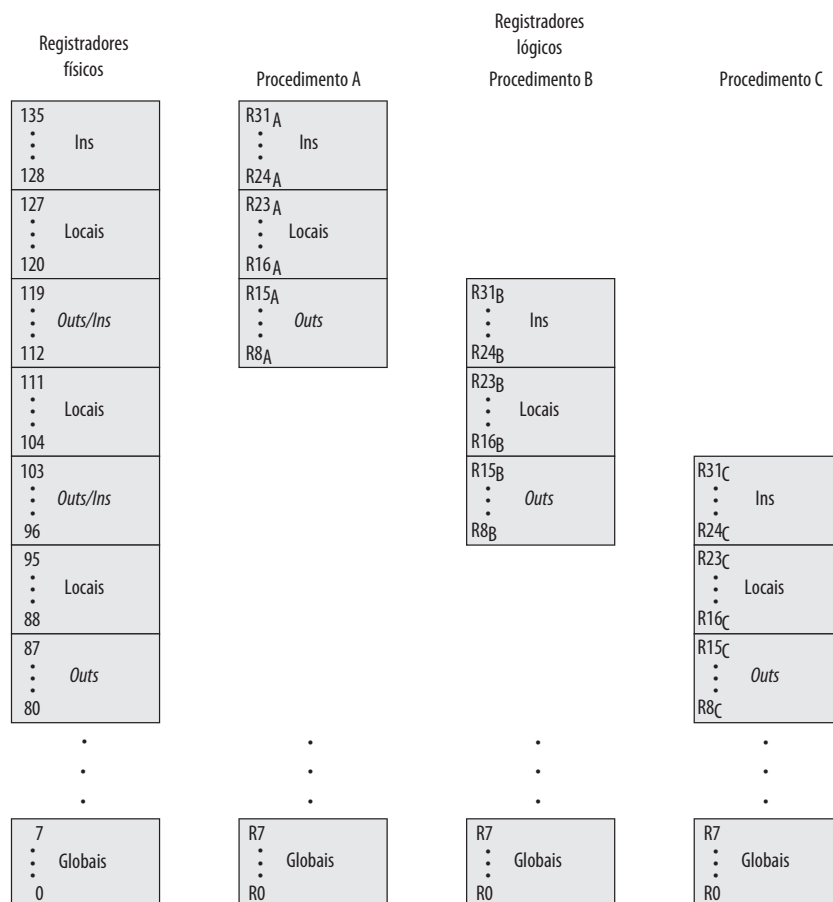
13.7 SPARC

A arquitetura SPARC (arquitetura de processador escalável — *scalable processor architecture*) refere-se a uma arquitetura definida pela Sun Microsystems. A Sun desenvolveu a sua própria implementação do SPARC, mas também licencia a arquitetura para outros fabricantes para produzirem máquinas compatíveis com SPARC. A arquitetura SPARC é inspirada na máquina RISC I de Berkeley e o seu conjunto de instruções e a sua organização de registradores é baseada no modelo RISC da Berkeley.

Conjunto de registradores do SPARC

Assim como acontece com o RISC de Berkeley, a SPARC também faz uso de janelas de registradores. Cada janela consiste de 24 registradores e o número total de janelas depende da implementação e varia de 2 a 32 janelas. A Figura 13.12 ilustra uma implementação que suporta 8 janelas, usando um total de 136 registradores físicos; conforme indica a discussão na Seção 13.2, este parece ser um número razoável de janelas. Registradores físicos de 0 a 7 são registradores globais compartilhados por todos procedimentos. Cada processador enxerga registradores lógicos de 0 a 31. Registradores lógicos de 24 a 31, referenciados como *ins*, são compartilhados com o procedimento-pai; e registradores lógicos de 8 a 15, referenciados como *outs*, são compartilhados com qualquer procedimento-filho. Estas duas partes se sobrepõem com outras janelas. Novamente, como indica a discussão na Seção 12.1, a disponibilidade de 8 registradores para passagem de parâmetros deveria ser adequada na maioria dos casos (como exemplo, veja a Tabela 13.4).

Figura 13.12 Layout das janelas de registradores SPARC com três procedimentos



A Figura 13.13 é outra visão de sobreposição de registradores. O procedimento que faz a chamada coloca quaisquer parâmetros a serem passados em seus registradores *outs*; o procedimento chamado trata esses mesmos registradores como seus registradores *ins*. O processador mantém um ponteiro da janela atual (CWP — *current window pointer*), localizado no registrador de *status* do processador (PSR — *processor status register*), o qual aponta para a janela do procedimento atualmente em execução. A máscara da janela inválida (WIM — *window invalid mask*), também em PSR, indica quais janelas estão inválidas.

Com a arquitetura SPARC de registradores, normalmente não é necessário salvar e restaurar registradores para uma chamada de procedimento. O compilador é simplificado porque ele precisa se preocupar apenas com a alocação de registradores locais para um procedimento de uma maneira eficiente e não precisa se preocupar com a alocação de registradores entre procedimentos.



Conjunto de instruções

A Tabela 13.11 lista as instruções para arquitetura SPARC. A maioria das instruções referencia apenas operandos de registradores. Instrução registrador-para-registrador possui três operandos e pode ser expressa desta forma

$$R_d \leftarrow R_{s1} \text{ op } S2$$

onde R_d e R_{s1} são referências de registradores; $S2$ pode se referir ou a um registrador ou a um operando imediato de 13 bits. Registrador zero (R_0) é definido no hardware com o valor 0. Esta forma é bem adaptada a programas típicos que possuem uma grande proporção de escalares e constantes locais.

As operações de ALU disponíveis podem ser agrupadas da seguinte forma:

- Adição de inteiros (com ou sem *carry*).
- Subtração de inteiros (com ou sem *carry*).
- Operadores booleanos AND, OR, XOR e suas negações.
- Deslocamento lógico à esquerda, lógico ou aritmético à direita.

Figura 13.13 Janelas com oito registradores formando uma pilha circular no SPARC

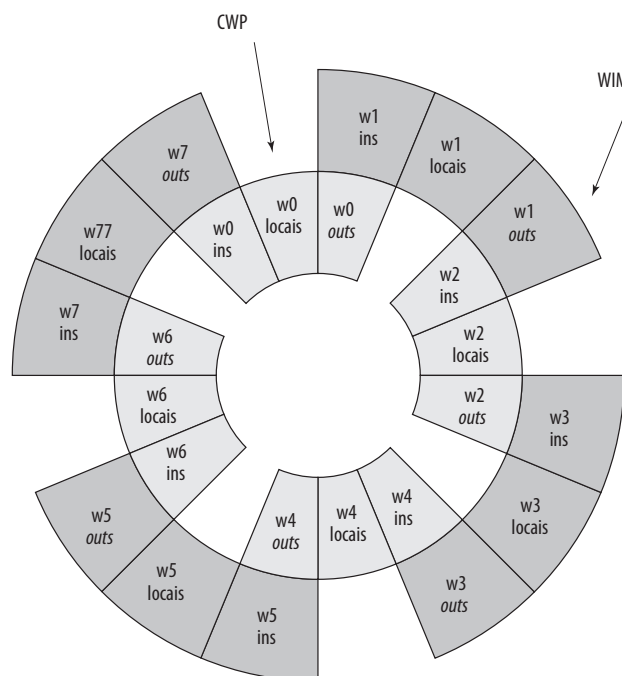


Tabela 13.11 Conjunto de instruções SPARC

OP	Descrição	OP	Descrição
Instruções Carregar/Armazenar		Instruções aritméticas	
LDSB	Carregar byte com sinal	ADD	Adicionar
LDSH	Carregar meia palavra com sinal	ADDCC	Adicionar, modificação do ICC
LDUB	Carregar byte sem sinal	ADDX	Adicionar com transporte
LDUH	Carregar meia palavra sem sinal	ADDXCC	Adicionar com transporte, modificação do ICC
LD	Carregar palavra	SUB	Subtrair
LDD	Carregar palavra dupla	SUBCC	Subtrair, modificação do ICC
STB	Armazenar byte	SUBX	Subtrair com <i>carry</i>
STH	Armazenar meia palavra	SUBXCC	Subtrair com transporte, modificação do ICC
STD	Armazenar palavra	MULSCC	Passo múltiplo, modificação do ICC
STDD	Armazenar palavra dupla	Instruções de salto/desvio	
Instruções de deslocamento		BCC	Desvio condicional
SLL	Deslocamento à esquerda lógico	FBCC	Desvio condicional de ponto flutuante
SRL	Deslocamento à direita lógico	CBCC	Desvio condicional do coprocessador
SRA	Deslocamento à direita aritmético	CALL	Chamada de procedimento
Instruções booleanas		JMPL	Saltar e ligar
AND	AND	TCC	Trap na condição
ANDCC	AND, modificação do ICC	SAVE	Avançar janela de registradores
ANDN	NAND	RESTORE	Mover janelas para trás
ANDNCC	NAND, modificação do ICC	RETT	Retornar de trap
OR	OR	Outras instruções	
ORCC	OR, modificação do ICC	SETHI	Modifica os 22 bits mais significativos
ORN	NOR	UNIMP	Instrução não implementada (trap)
ORNCC	NOR, modificação do ICC	RD	Ler um registrador especial
XOR	XOR	WR	Escrever um registrador especial
XORCC	XOR, modificação do ICC	IFLUSH	Esvaziar cache de instruções
XNOR	NOR exclusivo		
XNORCC	NOR exclusivo, modificação do ICC		

Todas estas instruções, exceto os deslocamentos, podem opcionalmente definir quatro código condicionais (ZERO, NEGATIVO, OVERFLOW, CARRY). Inteiros com sinal são representados em forma de dois complementos de 32 bits.

Apenas instruções simples de carregar e armazenar referenciam memória. Existem instruções de carregar e armazenar separadas para palavra (32 bits), palavra dupla, meia palavra e byte. Para dois últimos casos, existem instruções para carregar essas grandezas como números com ou sem sinal. Números com sinal são estendidos para preencher o registrador de destino de 32 bits. Números sem sinal são completados com zeros.

O único modo de endereçamento disponível além do modo de registrador é o modo por deslocamento. Isto é, o endereço efetivo (EA) de um operando consiste de um deslocamento a partir de um endereço contido em um registrador:

$$EA = (R_{s1}) + S2$$

ou

$$EA = (R_{s1}) + (R_{s2})$$

dependendo se o segundo operando é imediato ou uma referência de registrador. Para executar uma leitura ou escrita, um estágio extra é adicionado ao ciclo de instrução. Durante o segundo estágio, o endereço da memória é

calculado usando o ALU; leitura ou escrita ocorre no terceiro estágio. Este modo de endereçamento único é bem versátil e pode ser usado para sintetizar outros modos de endereçamento, conforme indicado na Tabela 13.12.

É instrutivo comparar a capacidade de endereçamento do SPARC com a do MIPS. O MIPS faz uso de um offset de 16 bits, comparado com o offset de 32 bits do SPARC. Por outro lado, o MIPS não permite que um endereço seja construído a partir dos conteúdos de dois registradores.



Formato da instrução

Assim como MIPS R4000, SPARC usa um conjunto simples de formatos da instrução de 32 bits (Figura 13.14). Todas as instruções começam com um *opcode* de 2 bits. Para maioria das instruções, isso é estendido com bits de *opcode* adicionais em outros lugares do formato. Para a instrução de Chamada (*call*), um operando de 30 bits é estendido com dois bits 0 para direita para formar um endereço relativo ao PC de 32 bits na forma de complemento de dois. As instruções são alinhadas dentro de um limite de 32 bits para que esta forma de endereçamento seja suficiente.

A instrução de Desvio inclui um campo condicional de 4 bits que corresponde ao padrão de 4 bits de código de condição, para que qualquer combinação de condições possa ser testada. O endereço relativo ao PC de 22 bits é estendido com dois zeros para a direita para formar um endereço de 24 bits em complemento de dois. Um recurso incomum da instrução Desvio é o bit de anulação. Quando o bit de anulação é igual a zero, a instrução depois do desvio é sempre executada, independentemente se o desvio é tomado. Esta é a típica operação de atraso de desvio encontrada em muitas máquinas RISC e descrita na Seção 13.5 (veja a Figura 13.7). No entanto, quando o bit de anulação é igual a um, a instrução depois do desvio só é executada se o desvio é tomado. O processador suprime o efeito dessa instrução mesmo que ela já esteja no pipeline. Esse bit de anulação é útil porque torna mais fácil para o compilador preencher o *delay slot* que segue um desvio condicional. A instrução que é alvo do desvio sempre pode ser colocada no *delay slot*, porque se o desvio não for tomado, a instrução pode ser anulada. Esta técnica é desejável porque os desvios condicionais geralmente são tomados mais da metade das vezes.

A instrução SETHI é uma instrução especial usada para carregar ou armazenar um valor de 32 bits. Este recurso é necessário para carregar e armazenar endereços e constantes grandes. A instrução SETHI define os 22 bits de ordem mais alta de um registrador com seu operando imediato de 22 bits e os 10 bits de ordem mais baixa com zeros. Uma constante imediata de até 13 bits pode ser especificada em um dos formatos gerais e tal instrução pode ser usada para preencher os 10 bits restantes do registrador. Uma instrução de carregar ou armazenar também pode ser usada para alcançar um modo de endereçamento direto. Para carregar um valor da posição K na memória, poderíamos usar as seguintes instruções SPARC:

```
sethi    %hi(K), %r8          ;carrega 22 bits de ordem mais alta do endereço da posição
                                ;K a registrador 8 r r
ld       [%r8 + %lo(K)], %r8   ;carrega conteúdo da posição K em r8
```

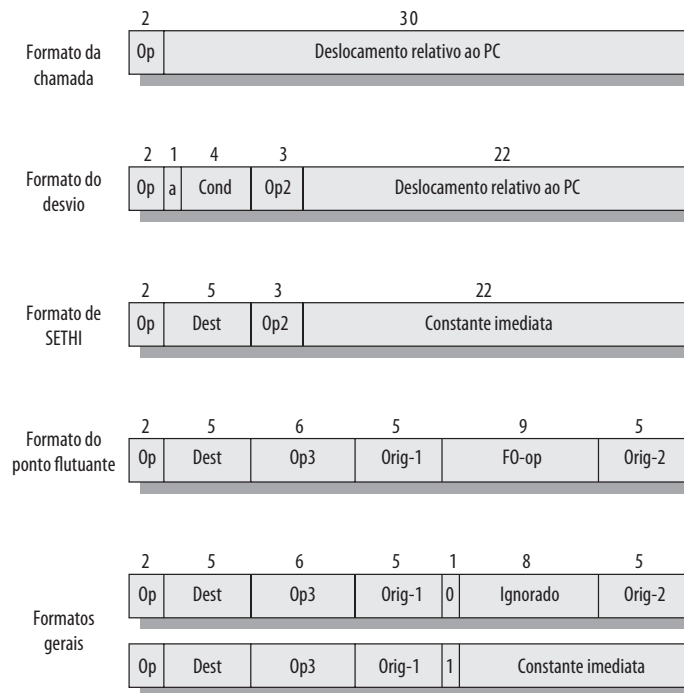
As macros %hi e %lo são usadas para definir operandos imediatos consistindo de bits de endereço apropriado de uma posição. Este uso de SETHI é semelhante ao uso da instrução LUI no MIPS.

O formato de ponto flutuante é usado para operações de ponto flutuante. Dois registradores de origem e um de destino são definidos.

Tabela 13.12 Sintetizando outros modos de endereçamento com modos de endereçamento do SPARC

Tipo de instrução	Modo de endereçamento	Algoritmo	Equivalente do SPARC
Registrador-para-registrador	Imediato	Operando = A	S2
Carregar, armazenar	Direto	EA = A	$R_0 + S2$
Registrador-para-registrador	Registrador	EA = R	$R_{s1} + R_{s2}$
Carregar, armazenar	Indireto de registrador	EA = (R)	$R_{s1} + 0$
Carregar, armazenar	Deslocamento	EA = (R) + A	$R_{s1} + S2$

S2 = pode ser um operando de registrador ou um operando imediato de 13 bits.

Figura 13.14 Formatos da instrução SPARC

Finalmente, todas as outras operações, incluindo carregar, armazenar, operações aritméticas e lógicas usam um dos dois últimos formatos mostrados na Figura 13.14. Um dos formatos usa dois registradores de origem e um registrador de destino, enquanto outro usa um registrador de origem, um operando de 13 bits imediato e um registrador de destino.



13.8 Controvérsia de RISC versus CISC

Durante muitos anos, a tendência geral na arquitetura e organização de computadores foi aumentar a complexidade do processador: mais instruções, mais modos de endereçamento, mais registradores especializados e assim por diante. O movimento RISC representa uma quebra fundamental com a filosofia por trás desta tendência. Naturalmente, o aparecimento de sistemas RISC e a publicação de artigos pelos seus proponentes exaltando as virtudes de RISC levaram a uma reação por parte daqueles envolvidos no projeto de arquiteturas CISC.

O trabalho que foi feito avaliando os méritos da abordagem RISC pode ser agrupado em duas categorias:

- **Quantitativa:** tenta comparar o tamanho do programa e a velocidade de execução dos programas em máquinas RISC e CISC que usam tecnologias comparáveis.
- **Qualitativa:** analisa questões como suporte para linguagem de alto nível e uso otimizado do estado atual de VLSI.

A maior parte do trabalho em avaliações quantitativas foi feito por aqueles que trabalham em sistemas RISC (PATTERSON e PIEPHO, 1982^p; HEATH, 1984^q; PATTERSON, 1984^x) e foi, em grande parte, favorável à abordagem RISC. Outros analisaram a questão e não ficaram convencidos (COLWELL et al., 1985^y; FLYNN, MITCHELL e MULDER, 1987^z; DAVIDSON e VAUGHAN, 1987^{aa}). Existem vários problemas ao se tentar fazer tais comparações (SERLIN, 1986^{bb}):

- Não existem pares de máquinas RISC e CISC que sejam comparáveis em custo de ciclo de vida, nível de tecnologia, sofisticação do compilador, suporte de sistema operacional e assim por diante.
- Não existe nenhum conjunto de programas de teste definitivo. Os desempenhos variam com o programa.
- É difícil separar os efeitos de hardware dos efeitos provenientes da capacidade em escrever compiladores.

- A maioria das análises comparativas sobre RISC foram feitas em máquinas “brinquedos” em vez de produtos comerciais. Além disso, a maioria das máquinas comercialmente disponíveis possui propagandas que as destacam como uma mistura de características RISC e CISC. Assim, uma comparação justa com uma máquina comercial real CISC (por exemplo, VAX, Pentium) é difícil.

A avaliação qualitativa é, quase que por definição, subjetiva. Vários pesquisadores voltaram a sua atenção para tal avaliação (COLWELL et al., 1985^y; WALLICH, 1985^{cc}), mas os resultados são, na melhor das hipóteses, ambíguos e certamente sujeitos à replicação (PATTERSON e HENNESSY, 1985^{dd}) e, é claro, à tréplica (COLWELL et al., 1985^{ee}).

Em anos mais recentes, a controvérsia RISC *versus* CISC sumiu quase que totalmente. Isso aconteceu porque houve uma convergência gradual das tecnologias. Com o aumento da densidade dos chips e da velocidade do hardware, os sistemas RISC se tornaram mais complexos. Ao mesmo tempo, em um esforço para obter o máximo desempenho, os modelos CISC focaram em questões tradicionalmente associadas com RISC, como um aumento no número de registradores de uso geral e mais ênfase no projeto do pipeline de instruções.



13.9 Leitura recomendada

Dois artigos clássicos sobre RISC são Patterson (1985^{ff}) e Henneessy (1984^g). Outro artigo de pesquisa é Stallings (1988^{gg}). Descrições de dois esforços pioneiros RISC são fornecidos por Radin (1983^h) e Patterson e Sequin (1982^a).

Kane e Heinrich (1992^{hh}) cobrem a máquina comercial MIPS em detalhes. Mirapuri, Woodacre e Vasseghi (1992ⁱⁱ) fornecem uma boa visão de MIPS R4000. Basheteen, Lui e Mullan (1991^{jj}) discutem a evolução do pipeline do R3000 para superpipeline do R4000. SPARC é coberto com bastantes detalhes em Dewar e Smosna (1990^{kk}).

Principais termos, perguntas de revisão e problemas

Principais termos

Computador com conjunto complexo de instruções (CISC)	Linguagem de alto nível (HLL)	Banco de registradores
Desvio atrasado	Computador com conjunto reduzido de instruções (RISC)	Janela de registradores
Leitura atrasada		SPARC

Perguntas de revisão

- 13.1 Quais são algumas das características peculiares típicas da organização RISC?
- 13.2 Explique brevemente duas abordagens básicas para minimizar operações registrador-memória em máquinas RISC.
- 13.3 Se um buffer circular de registradores é usado para tratar variáveis locais para procedimentos aninhados, descreva duas abordagens para lidar com variáveis globais.
- 13.4 Quais são algumas características típicas de uma arquitetura de conjunto de instruções RISC?
- 13.5 O que é um desvio atrasado?

Problemas

- 13.1 Considerando o padrão chamada-retorno da Figura 4.21, quantos *overflows* e *underflows* (dos quais cada um causa um salvar/restaurar do registrador) irão ocorrer com um tamanho de janela de:
 - a. 5?
 - b. 8?
 - c. 16?
- 13.2 Na discussão sobre a Figura 13.2, foi afirmado que apenas as duas primeiras partes de uma janela são salvas ou restauradas. Por que não é necessário salvar os registradores temporários?

- 13.3** Desejamos determinar o tempo de execução para um certo programa usando vários esquemas de pipeline discutidos na Seção 13.5. Seja

N = número de instruções executadas

D = número de acessos à memória

J = número de instruções de salto

Para o esquema sequencial simples (Figura 13.6a), o tempo de execução é $2N + D$ estágios. Desenvolva fórmulas para pipeline de dois, três e quatro estágios.

- 13.4** Reorganize a sequência de código da Figura 13.6d para reduzir número de NOOPs.

- 13.5** Considere o seguinte pedaço de código em uma linguagem de alto nível: for I in...100 loop

```
for I in 1...100 loop
  S ← S + Q(I).VAL
end loop;
```

Assuma que Q seja um vetor de registros de 32 bits e o campo VAL está nos 4 primeiros bytes de cada registro. Usando código x86, podemos compilar este pedaço de programa a seguir

```
MOV ECX, 1           ;usa registrador ECX para guardar I
LP: IMUL EAX, ECX, 32 ;obtem offset em EAX
    MOV EBX, Q[EAX]   ;carrega campo VAL
    ADD S, EBX         ;adiciona para S
    INC ECX           ;incrementa I
    CMP ECX, 101      ;compara com 101
    JNE LP            ;laço até I = 100
```

Este programa faz uso da instrução IMUL, a qual multiplica o segundo operando pelo valor imediato no terceiro operando e coloca o resultado no primeiro operando (veja o Problema 10.13). Um defensor de RISC gostaria de demonstrar que um compilador inteligente pode eliminar instruções complexas desnecessárias como IMUL. Forneça demonstração reescrevendo o programa x86 acima sem o uso da instrução IMUL.

- 13.6** Considere o seguinte laço:

```
S := 0;
for K := 1 to 100 do
  S := S - K;
```

Uma tradução direta disso para uma linguagem de montagem genérica ficaria parecida com algo assim:

```
LD R1, 0           ;guarda valor de S em R1
LD R2, 1           ;guarda valor de K em R2
LP SUB R1, R1, R2   ;S := S - K
    BEQ R2, 100, EXIT ;feito se K = 100
    ADD RE, RE, 1     ;senão incrementa K
    JMP LP            ;de volta para o início do laço
```

Um compilador para uma máquina RISC irá introduzir encaixes de atraso neste código para que o processador possa empregar o mecanismo de desvio atrasado. Instrução JMP é fácil de lidar, porque esta instrução é sempre seguida de instrução SUB; portanto, podemos simplesmente guardar uma cópia da instrução SUB em um *delay slot* depois de JMP. BEQ representa uma dificuldade. Não podemos deixar o código como está, porque a instrução ADD seria então executada muitas vezes a mais. Portanto, uma instrução NOP é necessária. Mostre o código resultante.

- 13.7** Uma máquina RISC pode fazer mapeamento de registradores simbólicos para registradores reais e também o rearranjo de instruções para eficiência do pipeline. Uma questão interessante surge relacionada à ordem em que essas duas instruções deveriam ser feitas. Considere o seguinte pedaço do programa:

```
LD SR1, A          ;carregar A no registrador simbólico 1
LD SR2, B          ;carregar B no registrador simbólico 2
ADD SR3, SR1, SR2  ;adicionar conteúdo de SR1 e SR2 e armazenar SR3
LD SR4, C
LD SR5, D
ADD SR6, SR4, SR5
```

- a. Faça primeiro o mapeamento de registradores e depois qualquer reordenação possível de instruções. Quantos registradores de máquinas são usados? Houve alguma melhoria do pipeline?

- b. Começando com o programa original, faça agora a reordenação de instruções e depois qualquer mapeamento possível. Quantos registradores de máquinas são usados? Houve alguma melhoria do pipeline?

13.8 Adicione entradas para os seguintes processadores na Tabela 13.7:

- a. Pentium II
- b. ARM

13.9 Em muitos casos, instruções de máquinas comuns que não estão listadas como parte do conjunto de instruções do MIPS podem ser sintetizadas com uma única instrução MIPS. Mostre isso para o seguinte:

- a. Mover de registrador para registrador.
- b. Incrementar, decrementar.
- c. Complementar.
- d. Negar.
- e. Esvaziar.

13.10 Uma implementação SPARC possui K janelas de registradores. Qual é o número N de registradores físicos?

13.11 SPARC não possui uma série de instruções comumente usadas em máquinas CISC. Algumas delas são facilmente simuladas usando ou registrador R0, o qual é sempre definido como 0, ou um operando constante. Essas instruções simuladas são chamadas de pseudoinstruções e são reconhecidas pelo compilador SPARC. Mostre como simular as seguintes pseudoinstruções, cada uma com uma única instrução SPARC. Em todas elas, *src* e *dst* se refere a registradores. (*Dica*: um armazenamento em R0 não tem nenhum efeito).

- a. MOV *src*, *dst*
- b. COMPARE *src*1, *src*2
- c. TEST *src*1
- d. NOT *dst*
- e. NEG *dst*
- f. INC *dst*
- g. DEC *dst*
- h. CLR *dst*
- i. NOP

13.12 Considere o seguinte pedaço de código:

```
if K > 10
    L := K + 1
else
    L := K - 1;
```

Uma tradução direta para *assembler* SPARC poderia ficar da seguinte forma:

sethi	%hi(K), %r8	;carrega 22 bits de ordem mais alta do endereço da posição
		;K no registrador r8
ld	[%r8 + %lo(k)] %r8	;carrega conteúdo da posição K em r8
cmp	%r8, 10	;compara conteúdo de r8 com 10
ble	L1	;desvio se (r8) ≤ 10
noop		
sethi	%hi(k), %r9	
ld	[%r9 + %lo(k)] %r9	;carrega conteúdo da posição K em r9
inc	%r9	;adiciona 1 para (r9)
sethi	%hi(L), %r10	
st	%r9, [%r10 + %lo(L)]	;armazena (r9) na posição L
b	L2	
nop		
L1: sethi	%hi(K), %r11	
ld	[%r11 + %lo(K)], %r12	;carrega conteúdo da posição K em r12
dec	%r12	;subtrai 1 de (r12)
sethi	%hi(L), %r13	
st	%r12, [%r13 + %lo(L)] + %lo(L)	;armazena (r12) na posição L
L2:		

O código contém um *nop* depois de cada instrução de desvio para permitir operação de desvio atrasado.

- a. Otimizações comuns de compiladores, que nada têm a ver com máquinas RISC, são geralmente eficientes em efetuar duas transformações no código acima mencionado. Observe que duas leituras são desnecessárias e que duas escritas podem ser combinadas se a escrita for movida para um lugar diferente no código. Mostre o programa após fazer essas duas alterações.
- b. Agora é possível fazer algumas otimizações específicas do SPARC. Nop depois de ble pode ser substituído movendo outra instrução para esse *delay slot* e definindo o bit de anulação na instrução ble (expresso como ble,a L1). Mostre o programa depois dessa mudança.
- c. Existem agora duas instruções desnecessárias. Remova-as e mostre o programa resultante.

Referências

- a PATTERSON, D. e SEQUIN, C. "A VLSI RISC". *Computer*, set. 1982.
- b LUNDE, A. "Empirical evaluation of some features of instruction set processor architectures". *Communications of the ACM*, mar. 1977.
- c HUCK, T. *Comparative analysis of computer architectures*. Stanford University Technical Report No. 83-243, mai. 1983.
- d TANENBAUM, A. "Implications of structured programming for machine architecture". *Communications of the ACM*, mar. 1978.
- e KATEVENIS, M. *Reduced instruction set computer architectures for VLSI*. Dissertação de PhD. Computer Science Department, University of California at Berkeley, outubro de 1983. Reimpresso por MIT Press, Cambridge, MA, 1985.
- f RAGAN-KELLEY, R. e CLARK, R. "Applying RISC theory to a large computer". *Computer Design*, nov. 1983.
- g TAMIR, Y. e SEQUIN, C. "Strategies for managing the register file in RISC". *IEEE Transactions on Computers*, nov. 1983.
- h CHAITIN, G. "Register allocation and spilling via graph coloring". *Proceedings, SIGPLAN Symposium on Compiler Construction*, jun. 1982.
- i CHOW, E. et al. "Engineering a RISC compiler system". *Proceedings, COMPCON Spring '86*, mar. 1986.
- j COUTANT, D.; HAMMOND, C.; e KELLEY, J. "Compilers for the new generation of Hewlett-Packard computers". *Proceedings, COMPCON Spring '86*, mar. 1986.
- k CHOW, F. e HENNESSY, J. "The priority-based coloring approach to register allocation". *ACM Transactions on Programming Languages*, out. 1990.
- l BRADLEE, D.; EGGERS, S. e HENRY, R. "The effect on risc performance of register set size and structure versus code generation strategy". *Proceedings, 18th Annual International Symposium on Computer Architecture*, mai. 1991.
- m HUGUET, M. e LANG, T. "Architectural support for reduced register saving/restoring in single-window register files". *ACM Transactions on Computer Systems*, fev. 1991.
- n HENNESSY, J., et al. "Hardware/software tradeoffs for increased performance". *Proceedings, Symposium on Architectural Support for Programming Languages and Operating Systems*, mar. 1982.
- o RADIN, G. "The 801 minicomputer". *IBM Journal of Research and Development*, mai. 1983.
- p PATTERSON, D. e PIEPHO, R. "Assessing RISCs in high-level language support". *IEEE Micro*, nov. 1982.
- q HEATH, J. "Re-evaluation of RISC 1". *Computer Architecture News*, mar. 1984.
- r MYERS, G. "The evaluation of expressions in a storage-to-storage architecture". *Computer Architecture News*, jun. 1978.
- s MASHEY, J. "CISC vs. RISC (or what is RISC really)". *USENET comp.arch newsgroup*, article 46782, fev. 1995.
- t BACON, F.; GRAHAM, S. e SHARP, O. "Compiler transformations for high-performance computing". *ACM Computing Surveys*, dez. 1994.
- u BRADLEE, D.; EGGERS, S. e HENRY, R. "Integrating register allocation and instruction scheduling for RISCs". *Proceedings, Fourth International Conference on Architectural Support for Programming Languages and Operating Systems*, abr. 1991.
- v HENNESSY, J. "VLSI processor architecture". *IEEE Transactions on Computers*, dez. 1984.
- w CHOW, E. et al. "How many addressing modes are enough?" *Proceedings, Second International Conference on Architectural Support for Programming Languages and Operating Systems*, out. 1987.
- x PATTERSON, D. "RISC watch". *Computer Architecture News*, mar. 1984.
- y COLWELL, R. et al. "Computers, complexity, and controversy". *Computer*, set. 1985.
- z FLYNN, M.; MITCHELL, C. e MULDER, J. "And now a case for more complex instruction sets". *Computer*, set. 1987.
- aa DAVIDSON, J. e VAUGHAN, R. "The effect of instruction set complexity on program size and memory performance". *Proceedings, Second International Conference on Architectural Support for Programming Languages and Operating Systems*, out. 1987.
- bb SERLIN, O. "MIPS, dhrystones, and other tales". *Datamation*, jun. 1986.
- cc WALLICH, P. "Toward simpler, faster computers". *IEEE Spectrum*, ago. 1985.
- dd PATTERSON, D. e HENNESSY, J. "Response to 'computers, complexity, and controversy'". *Computer*, nov. 1985.
- ee COLWELL, R. et al. "More controversy about 'computers, complexity, and controversy'". *Computer*, dez. 1985.
- ff PATTERSON, D. "Reduced instruction set computers". *Communications of the ACM*, jan. 1985.
- gg STALLINGS, W. "Reduced instruction set computer architecture". *Proceedings of the IEEE*, jan. 1988.
- hh KANE, G. e HEINRICH, J. *MIPS RISC architecture*. Englewood Cliffs, NJ: Prentice Hall, 1992.
- ii MIRAPURI, S.; WOODACRE, M. e VASSEGHI, N. "The MIPS R4000 processor". *IEEE Micro*, abr. 1992.
- jj BASHTEN, A., LUI, I. e MULLAN, J. "A superpipeline approach to the MIPS architecture". *Proceedings, COMPCON Spring '91*, fev. 1991.
- kk DEWAR, R. e SMOSNA, M. *Microprocessors: a programmer's view*. Nova York: McGraw Hill, 1990.